

Graph Machine Learning

Graph machine learning (Graph ML) learns from entities and the relationships between them. This guide develops the subject from graph representation through geometric deep learning, with interview-ready mathematics, runnable code, evaluation practices, and project ideas. Notation used throughout: a graph is $G = (V, E)$, $n = |V|$, $m = |E|$, node features are X , the adjacency matrix is A , and learned node states are H .

Graph Representation

1. Overview

A graph represents objects as **nodes** and relationships as **edges**. Unlike images or tables, graphs have no fixed ordering or grid: permuting node IDs must not change the meaning. Graph representations are useful when relationships carry predictive information, as in social networks, molecules, transactions, roads, recommender systems, program analysis, and knowledge bases.

2. Intuition

A table describes each person separately; a graph also records who knows whom. Two users with identical profiles may behave differently because one is connected to fraudsters and the other to trusted users. The graph supplies this relational context.

3. Prerequisites

- Sets, relations, matrices, sparse matrices, and basic linear algebra
- Python, NumPy, dictionaries, and complexity notation
- Probability and elementary ML feature engineering
- BFS/DFS and basic graph terminology

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Node/edge	Entity/relation	Defines the modeling unit	User/follows	What should become a node?
Directedness	Edge has or lacks orientation	Changes neighborhoods and walks	Follows vs friendship	How is directed A different?
Weight	Edge strength/cost	Preserves interaction intensity	Number of messages	Weight vs multiple edges
Features	Attributes on nodes, edges, or graph	Inputs to ML	Atom type, bond order	Structure-only fallback
Adjacency matrix	$A_{ij} \neq 0$ iff edge exists	Enables matrix operations	Social connections	Dense memory is $O(n^2)$
Edge list/CSR	Sparse storage	Scales with actual edges	(source, target) pairs	Why COO/CSR for GNNs?
Degree	Number/weight of incident edges	Centrality and normalization	Hub account	In/out-degree distinction
Path/component	Reachability structure	Determines information flow	Disconnected communities	What can message passing reach?
Subgraph	Graph restricted to selected nodes/edges	Sampling and local learning	Ego network	Induced vs sampled subgraph

Subtopic	Meaning	Why it matters	Example	Interview angle
Graph-level target	One label/value per graph	Molecular/property tasks	Toxicity of molecule	Node vs graph readout

5. Algorithm / Working Process

1. Decide what entities and relations answer the business question.
2. Assign stable node IDs; keep raw identifiers outside the model when sensitive.
3. Build edges with direction, timestamps, types, and weights when meaningful.
4. Construct node matrix X in $R^{(n \times d)}$, edge features, and targets.
5. Store structure as an edge list/COO or CSR sparse matrix.
6. Validate self-loops, duplicates, isolated nodes, leakage, and connectivity.
7. Choose a task: node-, edge-, or graph-level prediction.

Input is raw relational data; output is a consistent graph object plus features, labels, masks, and metadata.

6. Mathematical Foundation

For an unweighted directed graph:

$$A_{ij} = 1 \text{ if } (i, j) \text{ in } E, \text{ otherwise } 0$$

$$d_i(\text{out}) = \sum_j A_{ij}, \quad d_i(\text{in}) = \sum_j A_{ji}$$

$$D = \text{diag}(d_1, \dots, d_n)$$

For an undirected graph, $A = A^T$. The combinatorial Laplacian and normalized Laplacian are:

$$L = D - A$$

$$L_{\text{sym}} = I - D^{-1/2} A D^{-1/2}$$

$x^T L x = (1/2) \sum_{(i,j)} A_{ij} (x_i - x_j)^2$, so low graph-Laplacian energy means neighboring values vary smoothly. A node permutation matrix P changes storage but not the graph: $A' = P A P^T$, $X' = P X$.

7. Practical Implementation

```
import numpy as np

# Directed weighted graph: 0->1, 0->2, 2->1, 2->3, 3->0
edges = np.array([[0, 1], [0, 2], [2, 1], [2, 3], [3, 0]])
weights = np.array([1.0, 0.5, 2.0, 1.0, 0.7])
n = 4

A = np.zeros((n, n), dtype=float)
np.add.at(A, (edges[:, 0], edges[:, 1]), weights)
out_degree = A.sum(axis=1)
in_degree = A.sum(axis=0)

# Symmetrize for an undirected view and build normalized Laplacian.
A_u = np.maximum(A, A.T)
d = A_u.sum(axis=1)
inv_sqrt_d = np.divide(1.0, np.sqrt(d), out=np.zeros_like(d), where=d > 0)
L_sym = np.eye(n) - np.diag(inv_sqrt_d) @ A_u @ np.diag(inv_sqrt_d)

def neighbors(node):
    return edges[edges[:, 0] == node, 1].tolist()
```

```
assert neighbors(0) == [1, 2]
assert np.allclose(L_sym, L_sym.T)
print("A=\n", A, "\nout-degree=", out_degree, "\nin-degree=", in_degree)
```

For large graphs, use `scipy.sparse`, PyTorch Geometric `edge_index`, DGL, or a graph database instead of allocating dense `A`.

8. Code Explanation

`edges` is COO-like storage requiring $O(m)$ space. `np.add.at` correctly sums duplicate edges. Row and column sums give out- and in-degree. `maximum(A, A.T)` creates one undirected view; the diagonal degree factors normalize hub influence. The zero-degree-safe division prevents infinities for isolated nodes.

9. Training / Evaluation

Representation itself is not trained, but it controls every downstream experiment. Split data according to deployment time or entity boundaries before generating graph-derived features. Check label balance, missing features, duplicate/reverse edges, isolated-node rate, degree distribution, components, and train-test connectivity. Evaluate the eventual task with its appropriate metric and compare feature-only, structure-only, and combined baselines.

10. Complexity and Cost

Dense adjacency costs $O(n^2)$ memory; edge lists and CSR cost $O(n+m)$. Iterating neighbors costs $O(\deg(v))$ with adjacency lists. BFS/DFS is $O(n+m)$. Computing all-pairs shortest paths is expensive ($O(n^3)$ with Floyd-Warshall), so large systems use sampled or single-source methods. Sparse preprocessing is normally CPU-friendly.

11. Common Use Cases

- User-interaction and social graphs
- Molecules, proteins, and materials
- Product-user recommendation bipartite graphs
- Financial transaction and fraud networks
- Road, telecom, power, and supply-chain networks
- Knowledge graphs, dependency graphs, and code graphs

12. Common Mistakes

- Creating edges using information unavailable at prediction time
- Randomly splitting temporal graphs, causing future leakage
- Treating directed relations as undirected without justification
- Materializing dense adjacency for a sparse large graph
- Dropping edge type, time, multiplicity, or weight that carries signal
- Confusing zero padding with a real node/edge value
- Forgetting isolated nodes or duplicate edges
- Assuming node ID is a meaningful numeric feature

13. Edge Cases / Limitations

Graphs with no useful relational signal can underperform tabular models. High-degree hubs dominate naive aggregation; disconnected nodes receive no structural context. Dynamic, uncertain, privacy-sensitive, adversarial, or extremely heterogeneous relations need specialized modeling. A graph is also a modeling assumption: an incorrectly defined edge can inject bias more strongly than a noisy feature.

14. Variations

Variation	What changes	When to use	Importance
Multigraph	Multiple edges per pair	Repeated transactions	Projects/interviews

Variation	What changes	When to use	Importance
Temporal graph	Timestamped nodes/edges	Evolving interactions	High industry value
Hypergraph	One edge joins many nodes	Teams, sessions, reactions	Research
Signed graph	Positive/negative edges	Trust and antagonism	Specialized research
Multiplex graph	Several relation layers	Multi-channel networks	Advanced projects

15. Related Topics

Graph representation precedes embeddings and GNNs. Graph algorithms consume structure without learning; graph embeddings learn fixed vectors; GNNs learn task-conditioned vectors by message passing. Knowledge graphs emphasize typed facts, heterogeneous graphs multiple schemas, and geometric deep learning generalizes learning beyond Euclidean grids.

16. Interview Questions

- What is a graph?** A set of nodes V and edges E , optionally with directions, types, weights, timestamps, and features.
- Adjacency matrix vs list?** A matrix offers constant-time edge lookup but $O(n^2)$ memory; a list/CSR uses $O(n+m)$ and is preferred for sparse graphs.
- What makes graph data non-Euclidean?** Neighborhood sizes vary and nodes have no canonical order or regular coordinate grid.
- Why must a graph model be permutation invariant/equivariant?** Renaming nodes cannot change graph-level output; node outputs should be renamed correspondingly.
- What is degree?** Incident edge count or weight; directed graphs have separate in- and out-degree.
- Why add self-loops?** To include a node's current representation in neighborhood aggregation.
- What does the Laplacian encode?** Connectivity and smoothness; its spectrum reflects components and global structure.
- What is homophily?** Connected nodes tend to share labels/features; many classic GNNs exploit it.
- What is an induced subgraph?** It contains selected nodes and every original edge between them.
- How can graph splitting leak?** Test-time edges, future interactions, or the same entity appearing across splits can expose target information.
- How do you represent an isolated node?** Keep its feature row and ID; its structural representation must rely on self-features or external information.
- When is a graph unnecessary?** When relations are arbitrary, unavailable at inference, or add no signal over independent features.

17. Practice Tasks

- Code adjacency-list and CSR representations and benchmark neighbor lookup.
- Build a citation graph; report components, degree distribution, and isolated nodes.
- Compare directed, undirected, weighted, and unweighted versions downstream.
- Debug a graph whose validation accuracy collapses under a temporal split.
- Extend a simple graph to retain timestamps and edge types.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Transaction Graph Explorer	Finds hubs, components, suspicious cycles	Python, NetworkX, Plotly	Elliptic or synthetic payments	Data modeling + fraud insight
Road Network Analyzer	Computes routes and bottlenecks	OSMnx, NetworkX	OpenStreetMap	Algorithms + geospatial work
Citation Graph Builder	Converts papers/citations into ML-ready graph	Pandas, SciPy, PyG	Cora/OpenAlex subset	End-to-end graph pipeline

19. Quick Revision

- **Key idea:** model entities jointly with relationships.
- **Main formula:** $L = D - A$.
- **When to use:** relations influence the target.
- **Metrics:** representation checks plus downstream task metrics.
- **Traps:** leakage, dense storage, lost direction/type/time.
- **Interview one-liner:** "A graph is sparse, unordered relational data; its representation must preserve semantics while remaining permutation-safe."

20. Final Cheat Sheet

Item	Summary
Definition	$G=(V,E)$ with optional attributes
Input/output	Raw entities/relations -> nodes, edges, features, masks
Main steps	Define schema, construct, validate, split, store sparsely
Hyperparameters	Edge window/threshold, direction, weighting
Metrics	Degree/components/leakage checks + downstream metric
Pros/cons	Captures context / irregular, dependent, expensive
Best uses	Molecules, networks, recommendations, fraud, knowledge

Node Classification

1. Overview

Node classification predicts a label for each node using node attributes, graph structure, or both. Examples include classifying papers by subject, detecting fraudulent accounts, assigning protein function, and labeling products. It may be transductive (test nodes known during training) or inductive (unseen nodes arrive later), and single-label, multilabel, or multiclass.

2. Intuition

To classify a new employee's role, use their profile and the roles of people they collaborate with. Neighbor information helps when connected nodes are related, while the employee's own features protect against misleading neighbors.

3. Prerequisites

- Graph neighborhoods, adjacency, degree, and sparse storage
- Classification, logits, softmax/sigmoid, cross-entropy
- Train/validation/test splits and class imbalance
- PyTorch training loops and basic GNN message passing

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Transductive	All nodes/edges visible, some labels hidden	Learns within one fixed graph	Cora masks	Is using test-node features leakage?
Inductive	Predict unseen nodes/graphs	Matches live systems	New customer	Requires generalizable aggregator
Homophily	Neighbors share labels	Makes smoothing effective	Same-topic citations	What if heterophily is high?

Subtopic	Meaning	Why it matters	Example	Interview angle
Message passing	Aggregate neighbor states	Injects context	Mean of friend vectors	Permutation invariance
Label mask	Selects supervised nodes	Prevents target leakage	<code>train_mask</code>	Mask loss, not graph blindly
Imbalance	Rare classes have few nodes	Accuracy can mislead	Fraud nodes	Macro-F1/PR-AUC

5. Algorithm / Working Process

1. Input X , `edge_index/A`, node labels y , and split masks.
2. Produce node representations with feature-only ML, embeddings, or a GNN.
3. Map every node representation to class logits.
4. Compute loss only over labeled training nodes.
5. Backpropagate and select hyperparameters using validation nodes.
6. At inference, output argmax class or sigmoid probabilities for target nodes.

6. Mathematical Foundation

For C classes and logits z_i :

$$p(y_i=c \mid G, X) = \exp(z_{ic}) / \sum_k \exp(z_{ik})$$

$$L = -(1/|V_{\text{train}}|) \sum_{(i \in V_{\text{train}})} \log p(y_i \mid G, X)$$

Class-weighted loss uses $-w(y_i) \log p(y_i)$. A simple neighborhood model is:

$$h_i = \text{sigma}(W_{\text{self}} x_i + W_{\text{neigh}} * \text{mean}_{(j \in N(i))} x_j)$$

$$z_i = W_{\text{out}} h_i$$

For multilabel prediction, use one sigmoid per class and binary cross-entropy. Micro-F1 aggregates decisions globally; macro-F1 averages class F1 and exposes minority-class failure.

7. Practical Implementation

```
import torch
from torch import nn
import torch.nn.functional as F

# Tiny undirected graph. edge_index[0] sends to edge_index[1].
x = torch.tensor([[1., 0.], [0.9, 0.1], [0., 1.], [0.1, 0.9], [0.8, 0.2]])
edge_index = torch.tensor([[0,1,1,2,2,3,0,4,4,1],
                           [1,0,2,1,3,2,4,0,1,4]])
y = torch.tensor([0, 0, 1, 1, 0])
train_mask = torch.tensor([True, True, True, False, False])
test_mask = ~train_mask

class MeanGraphClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer = nn.Linear(4, 8)
        self.out = nn.Linear(8, 2)

    def forward(self, x, edge_index):
        src, dst = edge_index
        neigh = torch.zeros_like(x)
```

```

    neigh.index_add_(0, dst, x[src])
    degree = torch.bincount(dst, minlength=x.size(0)).clamp(min=1).unsqueeze(1)
    h = F.relu(self.layer(torch.cat([x, neigh / degree], dim=1)))
    return self.out(h)

model = MeanGraphClassifier()
optimizer = torch.optim.Adam(model.parameters(), lr=0.03, weight_decay=5e-4)
for _ in range(200):
    model.train(); optimizer.zero_grad()
    logits = model(x, edge_index)
    loss = F.cross_entropy(logits[train_mask], y[train_mask])
    loss.backward(); optimizer.step()

model.eval()
with torch.no_grad():
    pred = model(x, edge_index).argmax(1)
print("test predictions:", pred[test_mask].tolist())

```

8. Code Explanation

`index_add_` sums messages by destination without a graph library. `bincount` obtains destination degree and mean normalization; clamping handles isolated nodes. The model concatenates self and neighbor features so it need not erase a node's identity. Crucially, loss is indexed by `train_mask`; unlabeled test nodes can participate structurally in a transductive setting but their labels never do.

9. Training / Evaluation

Use stratified node splits only if the deployment setting is static; use time-, component-, or entity-based splits for evolving networks. Tune depth, hidden dimension, learning rate, dropout, weight decay, sampling fanout, and class weights. Report accuracy and macro-F1 for balanced multiclass tasks, PR-AUC/F1 for rare positives, ROC-AUC for ranking, and calibration when probabilities drive decisions. Compare MLP-only and graph-only baselines. Diagnose oversmoothing by layerwise embedding similarity and degree-sliced performance.

10. Complexity and Cost

A full-batch message-passing layer costs roughly $O(md + nd^2)$ for hidden width d and stores node activations plus edges. L layers can touch an exponentially growing receptive field under neighbor sampling. Inference may require the local L -hop subgraph. Small citation graphs run on CPU; millions of nodes usually need sparse GPU kernels, sampling, caching, or distributed storage.

11. Common Use Cases

- Fraudulent account or transaction-node detection
- Paper topic and document category prediction
- Protein function and cell type classification
- User interest, churn, or bot labeling
- Product taxonomy completion
- Network device anomaly classification

12. Common Mistakes

- Computing loss on validation/test labels
- Random splitting when edges or labels are temporal
- Claiming inductive ability from a transductive experiment
- Comparing only against GNNs and omitting an MLP baseline
- Using accuracy on severe imbalance
- Too many layers, causing oversmoothing or oversquashing
- Unnormalized aggregation lets hubs dominate

- Precomputing label-derived node features before the split

13. Edge Cases / Limitations

Low homophily can make neighbor averaging harmful. Isolated or new nodes rely on features. Long-range dependencies are compressed through narrow neighborhoods (oversquashing). Labels may be non-stationary, graph edges adversarial, or minority classes clustered in ways that break random validation. Transductive systems also need retraining or special handling when the graph changes.

14. Variations

Variation	Change and use	Importance
Label propagation	Diffuses known labels, no learned feature encoder	Essential baseline/interview
GCN/GAT/GraphSAGE	Convolution, attention, or sampled aggregation	Core placement topics
Heterophily GNN	Separates self/neighbor or mixes multi-hop signals	Advanced/research
Multilabel classifier	Sigmoid + BCE instead of softmax	Common industry case
Semi/self-supervised	Adds few-label or contrastive objectives	Projects/research

15. Related Topics

Node classification differs from graph classification, which predicts one label after pooling all nodes, and link prediction, which scores node pairs. Label propagation is non-parametric; an MLP ignores edges; GCN, GraphSAGE, and GAT learn relational representations. Community detection is unsupervised and its groups need not equal task labels.

16. Interview Questions

1. **What is node classification?** Predicting one or more labels per node from attributes and graph context.
2. **Transductive vs inductive?** Transductive training knows the fixed graph including unlabeled nodes; inductive models generalize to unseen nodes or graphs.
3. **Can test nodes appear during transductive training?** Their features/edges may, if the setting allows; their labels and future-only edges may not.
4. **Why does homophily help?** Neighbor aggregation then increases signal-to-noise because adjacent labels/features correlate.
5. **What loss is typical?** Masked cross-entropy for single-label multiclass; BCE-with-logits for multilabel.
6. **Why use macro-F1?** Each class contributes equally, revealing minority-class performance.
7. **What baseline is mandatory?** A feature-only MLP; it reveals whether the graph adds value.
8. **How are isolated nodes classified?** From self-features, learned ID embeddings, or external context.
9. **What is oversmoothing?** Deep propagation makes node representations increasingly indistinguishable.
10. **How do you handle imbalance?** Class-weighted/focal loss, careful sampling, threshold tuning, and PR-focused metrics.
11. **Why can random node splits overestimate quality?** Neighboring and temporally related nodes leak correlated information across splits.
12. **How would you debug a GNN below an MLP?** Check edge semantics, split leakage, homophily, normalization, self-loops, depth, and degree-sliced results.

17. Practice Tasks

- Implement masked cross-entropy and macro-F1 on a toy graph.
- Train MLP, label propagation, and GCN baselines on Cora.
- Plot accuracy by node degree and neighborhood homophily.
- Find and fix leakage from labels aggregated into node features.
- Extend single-label code to multilabel BCE and threshold tuning.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Fraud Account Classifier	Flags risky wallets using activity and neighbors	PyTorch Geometric, FastAPI	Elliptic	Imbalance + temporal graph skills
Paper Topic Predictor	Labels new papers and explains influential citations	PyG, MLflow	Cora/PubMed	Baselines + inductive evaluation
Bot Detection Graph	Combines profile and interaction structure	DGL/PyG, Pandas	Twibot-20	Real heterogeneous signals

19. Quick Revision

- **Key idea:** predict node labels from self and neighborhood information.
- **Main formula:** masked cross-entropy over V_{train} .
- **When to use:** target attaches to nodes and relations are predictive.
- **Metrics:** macro/micro-F1, accuracy, PR-AUC, calibration.
- **Traps:** transductive leakage, imbalance, oversmoothing.
- **Interview one-liner:** "Node classification is semi-supervised relational classification with a split protocol as important as the architecture."

20. Final Cheat Sheet

Item	Summary
Input/output	$X, A \rightarrow$ logits/probabilities for nodes
Main steps	Encode, aggregate, classify, masked loss
Hyperparameters	Depth, width, dropout, LR, fanout, class weights
Metrics	Accuracy, macro-F1, PR-AUC, ROC-AUC
Pros/cons	Uses context / can propagate noise and leakage
Best uses	Fraud, topics, proteins, users, taxonomy

Link Prediction

1. Overview

Link prediction estimates whether an edge should exist, will appear, or has a particular type. It powers recommendations, friend suggestions, knowledge-graph completion, biological interaction discovery, and missing-relationship detection. The central challenge is learning from a tiny set of observed positives amid an enormous and incompletely labeled space of node pairs.

2. Intuition

"Friends of friends often become friends" is a structural link predictor. ML generalizes this: encode each endpoint and its context, combine the encodings, and score compatibility. A missing edge is not automatically negative—it may simply be unobserved.

3. Prerequisites

- Graph paths, neighborhoods, embeddings, and sampling
- Binary/multiclass classification and ranking
- Dot products, sigmoid, BCE, margin losses
- ROC-AUC, PR-AUC, Hits@K, MRR, negative sampling

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Encoder	Maps nodes/subgraphs to vectors	Captures context	GNN embeddings	Shared vs separate encoders
Decoder	Scores a pair/relation	Expresses compatibility	Dot product	Symmetric vs asymmetric
Negative sampling	Samples unobserved pairs	All non-edges are too many	Corrupt destination	False negatives/bias
Temporal split	Train before validation/test	Matches future prediction	Purchases by date	Prevent edge leakage
Ranking	Orders candidates per query	Matches retrieval use	Top-10 products	MRR/Hits@K protocol
Enclosing subgraph	Local pair context	Captures paths/motifs	Common neighbors	Leakage from target edge

5. Algorithm / Working Process

1. Split positive edges before constructing training adjacency; for temporal systems, split by timestamp.
2. Build training graph from training edges only.
3. Encode nodes using features/structure.
4. Generate positive pairs and sampled negative pairs.
5. Decode each pair into a score and optimize BCE, margin, or ranking loss.
6. At inference, score candidate pairs, filter known invalid/seen edges, rank, and return top candidates.

6. Mathematical Foundation

Dot-product decoder and BCE:

```

s(u,v) = z_u^T z_v
p((u,v) in E) = sigmoid(s(u,v))
L = -sum_positive log sigmoid(s) - sum_negative log sigmoid(-s)

```

For relation r , a bilinear decoder is $s(u,r,v)=z_u^T R_r z_v$. Pairwise ranking can use $\max(0, \gamma - s_{\text{pos}} + s_{\text{neg}})$. Classical heuristics include:

```

CommonNeighbors(u,v) = |N(u) intersect N(v)|
Jaccard = |intersection| / |union|
AdamicAdar = sum_(w in intersection) 1/log(deg(w))

```

MRR is the mean reciprocal rank of the true target; Hits@K is its fraction ranked within K.

7. Practical Implementation

```

import torch
from torch import nn
import torch.nn.functional as F

torch.manual_seed(7)
n, dim = 6, 8
positive = torch.tensor([[0,0,1,2,3,4], [1,2,2,3,4,5]])
# Dot product is symmetric, so both orientations represent the same positive edge.
positive_set = set()
for u, v in positive.t().tolist():
    positive_set.update({(u, v), (v, u)})

```

```

def sample_negatives(count):
    pairs = []
    while len(pairs) < count:
        u, v = torch.randint(n, (2,)).tolist()
        if u != v and (u, v) not in positive_set:
            pairs.append((u, v))
    return torch.tensor(pairs).t()

class DotProductLP(nn.Module):
    def __init__(self, nodes, width):
        super().__init__()
        self.z = nn.Embedding(nodes, width)
    def forward(self, pairs):
        return (self.z(pairs[0]) * self.z(pairs[1])).sum(dim=1)

model = DotProductLP(n, dim)
optimizer = torch.optim.Adam(model.parameters(), lr=0.05)
for _ in range(300):
    negative = sample_negatives(positive.size(1))
    pairs = torch.cat([positive, negative], dim=1)
    labels = torch.cat([torch.ones(positive.size(1)), torch.zeros(negative.size(1))])
    optimizer.zero_grad()
    loss = F.binary_cross_entropy_with_logits(model(pairs), labels)
    loss.backward(); optimizer.step()

with torch.no_grad():
    candidates = torch.tensor([[0,1,3], [3,4,5]])
    print(torch.sigmoid(model(candidates)))

```

8. Code Explanation

The embedding table is a transductive encoder; a real inductive system would compute embeddings from node features and neighborhoods. The decoder is a symmetric dot product, appropriate for undirected compatibility but not directional relations. Each epoch resamples non-edges. `BCEWithLogits` is represented by its numerically stable functional equivalent, avoiding a separate sigmoid during training.

9. Training / Evaluation

Remove validation/test positives from the message-passing graph. Use temporal splits for future-edge prediction and entity-disjoint splits for unseen-node claims. Sample evaluation negatives consistently; results change dramatically with candidate set size and negative difficulty. Report ROC-AUC and especially PR-AUC for imbalance, plus MRR/Hits@K/Recall@K for retrieval. Tune embedding size, GNN depth, decoder, negative ratio, hard-negative strategy, margin, and regularization. Compare common-neighbor, popularity, matrix-factorization, and feature-only baselines.

10. Complexity and Cost

All possible pairs cost $O(n^2)$ and are rarely enumerated. Training with b sampled pairs costs decoder $O(bd)$ plus encoder cost. Exact top-K dot-product search is $O(nd)$ per query; approximate nearest-neighbor indexing reduces latency at some recall cost. Large embedding tables require $O(nd)$ memory and may dominate the GNN.

11. Common Use Cases

- Product, content, and people recommendation
- Social connection prediction
- Knowledge graph completion
- Drug-target and protein-protein interactions
- Missing dependency or citation discovery
- Fraud-ring and suspicious-transfer discovery

12. Common Mistakes

- Leaving test edges inside the training adjacency
- Treating every absent edge as a true negative
- Random edge splitting in a temporal product
- Sampling trivial negatives and reporting inflated AUC
- Evaluating with a different candidate space than deployment
- Using symmetric dot product for directed/typed relations
- Allowing duplicate or reverse positive pairs into negatives
- Reporting only accuracy on an overwhelmingly negative task

13. Edge Cases / Limitations

Cold-start nodes have no learned ID or history. Observed edges are selection-biased, and missing edges may be positives. Popularity can overwhelm personalization. Dynamic graphs invalidate cached embeddings. Pair scores may violate business constraints; post-filtering is required. A local GNN may miss long paths, while deep propagation risks oversmoothing and oversquashing.

14. Variations

Variation	What changes	When to use	Importance
Matrix factorization	Learns endpoint embeddings directly	Recommenders	Placement essential
GAE/VGAE	GNN encoder; deterministic/variational decoder	Attributed graphs	Strong project topic
SEAL	Classifies enclosing subgraphs	Motif-heavy links	Research/projects
Translational/bilinear KGE	Relation-aware score	Knowledge graphs	Essential for KG roles
Temporal LP	Time-conditioned encoder/decoder	Event streams	Industry/research

15. Related Topics

Link prediction is edge-level supervised/ranking learning; node classification predicts endpoint labels. Recommender retrieval is often bipartite link prediction. Knowledge-graph completion adds relation types and filtered ranking. Graph embeddings provide node vectors; GNN encoders make them task- and feature-dependent. Approximate nearest-neighbor search operationalizes large-scale inference.

16. Interview Questions

1. **What is link prediction?** Scoring candidate node pairs or triples for an existing/future relation.
2. **Why is it not ordinary balanced classification?** Potential non-edges grow as $O(n^2)$ and most are unlabeled rather than confirmed negatives.
3. **What is negative sampling?** Training on selected unobserved/corrupted pairs instead of every non-edge.
4. **Why remove test edges before encoding?** Otherwise the answer directly affects node embeddings—a structural leakage.
5. **Dot product limitation?** It is symmetric, so it cannot model antisymmetric directed relations by itself.
6. **ROC-AUC vs PR-AUC?** PR-AUC is more informative when positives are rare; ROC-AUC may look strong despite poor precision.
7. **MRR meaning?** Average $1/\text{rank}$ of the true answer across queries.
8. **What is filtered KG evaluation?** Other known true triples are removed from corrupt candidates before rank calculation.
9. **How handle cold start?** Use attributes, inductive GNNs, content encoders, or onboarding signals.
10. **What is a hard negative?** A plausible but incorrect candidate close to the positive under structure/model score.
11. **Why can random negative sampling inflate results?** Random pairs are often obviously unrelated, unlike production candidates.
12. **How serve top-K links?** Precompute embeddings, use ANN retrieval, apply constraints, then rerank with richer features.

17. Practice Tasks

- Implement common-neighbor, Jaccard, and Adamic-Adar baselines.
- Build a GAE on Cora with a leakage-safe edge split.
- Compare uniform and hard negatives under PR-AUC and Hits@10.
- Debug a suspicious 0.999 AUC caused by retained test edges.
- Extend the dot-product decoder to a directed bilinear score.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Citation Recommender	Suggests likely relevant citations	PyG, FAISS	Cora/OGB-Citation2	Retrieval + graph evaluation
Drug Interaction Predictor	Ranks possible drug-drug interactions	PyTorch, RDKit	DrugBank subset	High-impact scientific ML
E-commerce Candidate Generator	Retrieves user-product candidates	LightGCN, FastAPI, FAISS	MovieLens/Amazon	Production recommendation stack

19. Quick Revision

- **Key idea:** encode endpoints, decode compatibility, rank candidates.
- **Main formula:** $p(u,v) = \text{sigmoid}(z_u^T z_v)$.
- **When to use:** target is a missing/future edge.
- **Metrics:** PR-AUC, MRR, Hits/Recall@K.
- **Traps:** test-edge leakage, false/easy negatives, wrong candidate set.
- **Interview one-liner:** "Link prediction is extreme-imbalance ranking over pairs, so split and negative-sampling protocols define the validity of the result."

20. Final Cheat Sheet

Item	Summary
Input/output	Graph + candidate pairs -> link scores/ranks
Main steps	Split edges, encode, sample negatives, decode, rank
Hyperparameters	Dimension, depth, negative ratio, margin, top-K
Metrics	PR-AUC, ROC-AUC, MRR, Hits@K, Recall@K
Pros/cons	Discovers relations / quadratic candidates, uncertain negatives
Best uses	Recommendations, KG completion, biological links

Graph Embeddings

1. Overview

Graph embeddings map nodes, edges, subgraphs, or entire graphs into low-dimensional vectors that preserve useful structure or semantics. They convert irregular graph objects into fixed-size features usable by classifiers, clustering, retrieval, visualization, and recommendation systems. Classical embeddings are usually shallow lookup vectors; modern GNN embeddings are functions of features and neighborhoods.

2. Intuition

A city map is complex, but coordinates let us calculate proximity. A graph embedding gives each node coordinates so nodes that are structurally or semantically related lie near one another. "Near" depends on the objective: immediate adjacency, shared neighborhoods, random-walk co-occurrence, roles, or graph-level similarity.

3. Prerequisites

- Linear algebra: eigendecomposition, SVD, dot products, norms
- Probability, random walks, softmax, negative sampling
- Unsupervised learning, PCA, clustering, nearest neighbors
- Basic graph structure and node/link prediction

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Proximity	Relationship the vectors preserve	Defines embedding semantics	First/second-order	What does similarity mean?
Shallow embedding	One learned vector per node	Fast, strong transductive baseline	Node2Vec	Cannot encode unseen nodes
Encoder	Function generating vector	Enables inductive use	GraphSAGE	Lookup vs functional encoder
Decoder/objective	Reconstructs relation from vectors	Shapes geometry	Dot product adjacency	Objective-task alignment
Graph pooling	Combines node vectors	Gives graph-level embedding	Mean/sum/readout	Sum vs mean expressiveness
Similarity	Compares embeddings	Drives retrieval/clustering	Cosine/dot/Euclidean	Normalization effects

5. Algorithm / Working Process

1. Select an embedding unit: node, edge, subgraph, or whole graph.
2. Define similarity to preserve: adjacency, walks, matrix factorization, labels, or task loss.
3. Choose an encoder: lookup, spectral method, matrix factorization, or GNN.
4. Optimize reconstruction, contrastive, skip-gram, or supervised loss.
5. Validate with downstream tasks and task-independent probes.
6. Store/version vectors and use them for retrieval, classification, clustering, or visualization.

6. Mathematical Foundation

A generic reconstruction objective is:

$$\min_Z \sum_{(i,j)} \text{loss}(S_{ij}, \text{decoder}(z_i, z_j)) + \lambda \|Z\|_F^2$$

where S may be adjacency, co-occurrence, or another similarity. Spectral embedding uses eigenvectors of L :

$$\min_Z \text{Tr}(Z^T L Z), \text{ subject to } Z^T Z = I$$

Graph-level mean pooling is $z_G = (1/n) \sum_i h_i$; sum pooling preserves graph size and is more expressive for multisets. Cosine similarity is $(z_i^T z_j) / (\|z_i\| \|z_j\|)$. Contrastive methods increase agreement for positive views and decrease it for negatives, often via InfoNCE.

7. Practical Implementation

```
import numpy as np
from sklearn.cluster import KMeans
```

```
# Spectral node embedding of two weakly connected triangles.
A = np.array([
    [0,1,1,0,0,0], [1,0,1,0,0,0], [1,1,0,.1,0,0],
    [0,0,.1,0,1,1], [0,0,0,1,0,1], [0,0,0,1,1,0]
], dtype=float)
d = A.sum(1)
D_inv_sqrt = np.diag(1 / np.sqrt(d))
L = np.eye(len(A)) - D_inv_sqrt @ A @ D_inv_sqrt

eigenvalues, eigenvectors = np.linalg.eigh(L) # symmetric solver
Z = eigenvectors[:, 1:3] # skip near-constant first eigenvector
labels = KMeans(n_clusters=2, n_init=10, random_state=0).fit_predict(Z)

assert len(np.unique(labels[:3])) == 1
assert len(np.unique(labels[3:])) == 1
print("embedding=\n", Z, "\nclusters=", labels)
```

8. Code Explanation

The normalized Laplacian represents graph smoothness. Its smallest nontrivial eigenvectors vary slowly across strong edges, separating the weakly connected triangles. `eigh` is numerically appropriate for a real symmetric matrix. This dense demonstration is pedagogical; production spectral methods use sparse eigensolvers.

9. Training / Evaluation

Separate embedding training from downstream evaluation to avoid label leakage. Use a fixed linear classifier ("linear probe") for node embeddings, link prediction with held-out edges, clustering NMI/ARI, retrieval Recall@K, or graph classification. Compare against degree/handcrafted features and random embeddings. Evaluate stability across seeds, dimensionalities, and time. For inductive claims, hold out entire nodes or graphs before fitting the encoder.

10. Complexity and Cost

A lookup table costs $O(nd)$ memory. Dense eigendecomposition is $O(n^3)$ and infeasible for large graphs; truncated sparse methods depend on m , requested eigenvectors, and convergence. Random-walk methods cost walk generation plus skip-gram training. GNN embeddings cost message passing and activations. Vector retrieval may require an ANN index with additional memory.

11. Common Use Cases

- Candidate retrieval and recommendation
- Node clustering and community discovery
- Features for node/link/graph classifiers
- Similarity search for molecules or entities
- Graph visualization after 2-D projection
- Anomaly detection by embedding distance

12. Common Mistakes

- Saying embeddings "preserve the graph" without defining proximity
- Evaluating on edges/nodes used by an unsupervised objective
- Using t-SNE plots as primary quantitative evidence
- Comparing cosine and dot product without considering vector norms
- Claiming inductive generalization for a node lookup table
- Choosing dimension only by downstream test performance
- Ignoring embedding drift and node-ID alignment across retrains

13. Edge Cases / Limitations

No low-dimensional geometry preserves every graph property. Hubs, disconnected components, bipartite structure, heterophily, and rare roles can distort proximity. Shallow methods cannot represent unseen nodes and memory grows with node count. Embeddings may encode sensitive attributes and are often difficult to explain. Rotational invariance means independently trained spaces are not directly coordinate-aligned.

14. Variations

Variation	What changes	When to use	Importance
Spectral	Laplacian eigenvectors	Clustering/small graphs	Interview math
Matrix factorization	Factorizes adjacency/similarity	Strong shallow baseline	Placements
Random-walk	Skip-gram on node sequences	Large transductive graphs	Core
GNN embedding	Neighborhood encoder	Features/unseen nodes	Core industry
Graph2Vec/readout	One vector per graph	Molecules/documents	Projects
Hyperbolic	Non-Euclidean latent space	Hierarchical graphs	Research

15. Related Topics

DeepWalk and Node2Vec are random-walk node embeddings. Matrix factorization makes the reconstruction view explicit. GCN/GraphSAGE/GAT produce contextual embeddings optimized end-to-end. Knowledge-graph embeddings model typed triples. PCA embeds tabular vectors; graph embeddings must additionally decide which relational proximity to preserve.

16. Interview Questions

- What is a graph embedding?** A vector representation of a graph object preserving task-relevant structure or semantics.
- What can be embedded?** Nodes, edges, subgraphs, paths, relations, or entire graphs.
- Shallow vs GNN embeddings?** Shallow methods learn per-ID vectors; GNNs compute vectors from features and neighborhoods.
- First-order proximity?** Direct edge closeness; second-order means similar neighborhood distributions.
- Why skip the first Laplacian eigenvector?** For a connected graph it is trivial/constant under the appropriate normalization.
- How evaluate embeddings?** Downstream probes, link ranking, clustering, retrieval, and stability—not visualization alone.
- Why are coordinates non-identifiable?** Rotations/reflections can preserve all pairwise dot products or distances.
- When use cosine over dot product?** When direction should matter but magnitude should not.
- How get edge embeddings?** Combine endpoints via concatenation, difference, Hadamard product, or a learned decoder.
- What is graph pooling?** A permutation-invariant readout over node states producing a graph vector.
- Why do shallow embeddings fail cold start?** An unseen ID has no optimized vector.
- Can higher dimension always help?** No; it increases cost and overfitting and may preserve irrelevant structure.

17. Practice Tasks

- Implement Laplacian eigenmaps and visualize Zachary's Karate Club.
- Compare spectral, DeepWalk, and GCN embeddings with a fixed linear probe.
- Test cosine, dot product, and Euclidean retrieval.
- Diagnose accidental label leakage in supervised embedding evaluation.
- Align two retrained spaces with orthogonal Procrustes and measure drift.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Graph Embedding Benchmark	Fairly compares four encoders	PyG, scikit-learn, MLflow	Cora/BlogCatalog	Experimental rigor
Molecule Similarity Search	Retrieves structurally related molecules	RDKit, GIN, FAISS	ZINC/ESOL	Scientific retrieval

Project	What it does	Stack	Dataset	Resume value
Network Anomaly Map	Finds embedding outliers and explains neighborhoods	NetworkX, PyTorch, Plotly	Elliptic	Graph analytics + visualization

19. Quick Revision

- **Key idea:** compress relational structure into task-useful vectors.
- **Main formula:** reconstruct similarity S_{ij} from z_i, z_j .
- **When to use:** conventional ML/retrieval needs fixed-size graph features.
- **Metrics:** linear-probe score, MRR/Recall@K, NMI/ARI.
- **Traps:** undefined proximity, leakage, transductive cold start.
- **Interview one-liner:** "An embedding is useful only relative to the graph similarity and downstream task its objective preserves."

20. Final Cheat Sheet

Item	Summary
Definition	Low-dimensional vectors for graph objects
Input/output	Graph/features -> node/edge/graph vectors
Main steps	Define proximity, encode, optimize, probe
Hyperparameters	Dimension, context, negatives, pooling
Metrics	Probe accuracy/F1, MRR, Recall@K, NMI
Pros/cons	ML-friendly compact features / information loss, drift
Best uses	Retrieval, clustering, prediction, visualization

DeepWalk

1. Overview

DeepWalk is an unsupervised, transductive node-embedding algorithm that runs truncated random walks and treats node sequences like sentences. A Skip-gram model learns vectors that predict nearby nodes in those walks. It was influential because it transferred distributional representation learning from language to large graphs and connects random-walk co-occurrence to matrix factorization.

2. Intuition

If two people repeatedly appear in the same short walks through a social network, they occupy nearby network regions, much as words appearing in similar textual contexts acquire similar vectors. Walks provide many local "sentences" without node labels.

3. Prerequisites

- Uniform random walks and Markov chains
- Word2Vec Skip-gram, softmax, negative sampling
- Graph embeddings and adjacency lists
- Basic SGD and cosine similarity

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Uniform walk	Next neighbor chosen uniformly	Samples local topology	$v_1, v_4, v_2, \dots$	DeepWalk vs Node2Vec
Context window	Nodes near center in a walk	Defines learned proximity	+/- 5 positions	Dynamic window effect
Skip-gram	Predict context from center	Learns embeddings	$P(v_{\text{context}} v_{\text{center}})$	
Negative sampling	Contrasts random nodes	Makes training scalable	5 negatives/positive	Noise distribution
Walk corpus	Many walks from every node	Controls coverage	10 walks/node	Bias from degree
Transductive table	Vector per training node	Simple and fast	$Z[\text{node_id}]$	Cold-start limitation

5. Algorithm / Working Process

1. Store each node's neighbor list.
2. For every node, generate several length- L uniform random walks.
3. Slide a context window over every walk to create center-context pairs.
4. Train Skip-gram with hierarchical softmax or negative sampling.
5. Use input embeddings as node vectors.
6. Feed vectors into a classifier, clusterer, or link decoder.

6. Mathematical Foundation

Full Skip-gram maximizes:

$$\sum_{(u \text{ in walks})} \sum_{(v \text{ in context}(u))} \log P(v|u)$$

$$P(v|u) = \exp(z'_v{}^T z_u) / \sum_w \exp(z'_w{}^T z_u)$$

Negative sampling replaces the expensive denominator:

$$\log \text{sigmoid}(z'_v{}^T z_u) + \sum_{(k=1..K)} E_{(n \sim P_n)} [\log \text{sigmoid}(-z'_n{}^T z_u)]$$

Walk transition is $P(v_{(t+1)}=x \mid v_t=v)=1/\text{deg}(v)$ for $x \text{ in } N(v)$. In expectation, DeepWalk factorizes a transformation related to sums of powers of the random-walk transition matrix, so window length determines how many-hop proximity it captures.

7. Practical Implementation

```
import random
import torch
from torch import nn
import torch.nn.functional as F

random.seed(0); torch.manual_seed(0)
neighbors = {0:[1,2], 1:[0,2], 2:[0,1,3], 3:[2,4,5], 4:[3,5], 5:[3,4]}

def walk(start, length=8):
    result = [start]
    while len(result) < length and neighbors[result[-1]]:
        result.append(random.choice(neighbors[result[-1]]))
    return result

walks = [walk(v) for _ in range(20) for v in neighbors]
```

```

pairs, window = [], 2
for sequence in walks:
    for i, center in enumerate(sequence):
        for j in range(max(0, i-window), min(len(sequence), i+window+1)):
            if i != j:
                pairs.append((center, sequence[j]))

centers, contexts = torch.tensor(pairs).t()
class SkipGram(nn.Module):
    def __init__(self, n, d):
        super().__init__()
        self.center = nn.Embedding(n, d)
        self.context = nn.Embedding(n, d)
    def loss(self, u, v, negatives=5):
        pos = (self.center(u) * self.context(v)).sum(1)
        neg_ids = torch.randint(len(neighbors), (len(u), negatives))
        neg = torch.einsum("bd,bkd->bk", self.center(u), self.context(neg_ids))
        return -(F.logsigmoid(pos) + F.logsigmoid(-neg).sum(1)).mean()

model = SkipGram(len(neighbors), 8)
opt = torch.optim.Adam(model.parameters(), lr=.03)
for _ in range(150):
    idx = torch.randint(len(centers), (64,))
    opt.zero_grad(); loss = model.loss(centers[idx], contexts[idx]); loss.backward(); opt.step()

Z = F.normalize(model.center.weight.detach(), dim=1)
print("similarity(0,1):", float(Z[0] @ Z[1]))

```

8. Code Explanation

Uniform walks create topology-derived sequences. The window emits multiple positive context pairs per position. Separate center and context tables match Skip-gram. `logsigmoid` implements stable negative-sampling loss, and `einsum` scores all negatives in a batch. The final normalized center table supports cosine similarity. Production code should batch all pairs, use degree-based noise, and avoid Python walk bottlenecks.

9. Training / Evaluation

Generate walks only from the graph allowed by the split. Tune dimension, walks per node, walk length, window, negatives, epochs, and noise distribution. For node classification, freeze embeddings and train a logistic regression on training labels; report macro-F1. For link prediction, exclude held-out edges before walks and report PR-AUC/MRR. Repeat seeds because walks and SGD are stochastic. Compare degree features, spectral embeddings, Node2Vec, and a GNN when attributes exist.

10. Complexity and Cost

Walk generation costs $O(n * r * L)$ for r walks/node and length L . Corpus storage has the same order unless streamed. Skip-gram cost is approximately $O(\text{number_of_pairs} * (K+1) * d)$. Embeddings cost $O(nd)$ memory, often the dominant term. Training can run on CPU for moderate graphs and parallelizes across walks.

11. Common Use Cases

- Social/community node features
- Citation and co-authorship classification
- Candidate generation and similarity search
- Structure-only clustering
- Initialization for downstream graph models
- Anomaly analysis through neighborhood embeddings

12. Common Mistakes

- Running walks over validation/test edges for link prediction

- Treating node IDs as ordinal values
- Not shuffling start nodes or training pairs
- Too-short walks/windows that capture only adjacency
- Too-long contexts that blur communities
- Ignoring disconnected or low-degree nodes
- Claiming unseen-node inference
- Evaluating on the same relations that generated contexts

13. Edge Cases / Limitations

DeepWalk ignores node and edge attributes, direction semantics, types, and timestamps unless the walk mechanism is adapted. It favors community proximity rather than structural roles. High-degree nodes dominate visits, small components supply repetitive contexts, and unseen nodes require retraining. Uniform walks provide less control than Node2Vec.

14. Variations

Variation	Change	When to use	Importance
Hierarchical softmax	Tree-based output instead of negatives	Large vocabularies	Historical/interview
Node2Vec	Biased second-order walks	Tune local/global exploration	Core
Walklets	Skips positions in walks	Explicit multi-scale proximity	Research/project
Metapath2Vec	Type-constrained walks	Heterogeneous graphs	Important extension
Temporal walks	Respect edge time	Dynamic networks	Industry/research

15. Related Topics

Word2Vec supplies the Skip-gram objective; matrix factorization explains what the learned co-occurrence geometry approximates. Node2Vec changes the walk distribution with p, q . GraphSAGE replaces ID lookups with feature aggregation for inductive learning. Personalized PageRank also uses random walks but produces diffusion scores rather than a shared embedding space.

16. Interview Questions

1. **What is DeepWalk?** Uniform random walks plus Skip-gram for unsupervised node embeddings.
2. **Why call walks sentences?** Nodes are tokens and nearby walk positions define context co-occurrence.
3. **Is it supervised?** The embedding objective is self-supervised/unsupervised; labels are not needed.
4. **Why negative sampling?** It avoids a full softmax over every node.
5. **What proximity is preserved?** Random-walk co-occurrence, mixing local and higher-order community structure.
6. **Main hyperparameters?** Dimension, walks/node, walk length, window, negatives, epochs.
7. **Is DeepWalk inductive?** No; it learns a table for known node IDs.
8. **How differs from Node2Vec?** DeepWalk walks uniformly; Node2Vec biases return and outward movement.
9. **How prevent LP leakage?** Remove held-out positive edges before generating walks.
10. **Effect of larger window?** Captures broader context but may blur local distinctions.
11. **Why two embedding tables?** Skip-gram separately models a node as center and as context; usually one table is exported.
12. **What is its matrix-factorization connection?** Skip-gram on walks implicitly factorizes a shifted/log-transformed walk co-occurrence matrix.

17. Practice Tasks

- Implement walk generation and verify empirical next-step probabilities.
- Train DeepWalk on Karate Club and classify club membership.
- Sweep window size and measure community separation.
- Debug link leakage caused by generating walks before edge splitting.
- Stream walks instead of storing the complete corpus.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
DeepWalk from Scratch	Implements and benchmarks every stage	PyTorch, NetworkX	Karate/Cora	Shows fundamentals
Citation Similarity Search	Finds structurally related papers	Gensim/PyTorch, FAISS	Cora	Representation + retrieval
Community Drift Monitor	Retrains embeddings and tracks aligned clusters	Python, Procrustes, Plotly	Reddit snapshots	Temporal monitoring insight

19. Quick Revision

- **Key idea:** random walks create sentences; Skip-gram learns node vectors.
- **Main formula:** positive log-sigmoid plus sampled-negative log-sigmoid.
- **When to use:** structure-only, fixed graph, scalable baseline.
- **Metrics:** macro-F1, MRR/PR-AUC, clustering NMI.
- **Traps:** transductive claims and held-out-edge leakage.
- **Interview one-liner:** "DeepWalk learns community-aware node vectors by predicting random-walk context with Skip-gram."

20. Final Cheat Sheet

Item	Summary
Input/output	Graph -> lookup embedding per known node
Main steps	Uniform walks, windows, Skip-gram, export vectors
Hyperparameters	d , walks, length, window, negatives
Metrics	Probe F1, LP MRR/PR-AUC, NMI
Pros/cons	Simple/scalable / no features, types, cold start
Best uses	Community-oriented fixed-graph embeddings

Node2Vec

1. Overview

Node2Vec extends DeepWalk with a second-order biased random walk controlled by return parameter p and in-out parameter q . It can interpolate between breadth-first-like exploration that preserves local communities and depth-first-like exploration that captures broader structural roles. The generated sequences still train a Skip-gram model.

2. Intuition

At node v , the next move depends on where the walker came from, t . A cautious walker stays near home and learns neighborhoods; an exploratory walker travels outward and may discover nodes playing similar roles in different regions. p controls returning to t ; q controls staying near t versus moving away.

3. Prerequisites

- DeepWalk, Skip-gram, and negative sampling
- Conditional probability and second-order Markov walks
- BFS vs DFS intuition and graph distance
- Embedding evaluation and leakage-safe link splits

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Second-order walk	Transition depends on previous node	Enables exploration bias	$t \rightarrow v \rightarrow x$	Why not first-order?
Return p	Penalizes immediate backtracking	Controls revisits	weight $1/p$	Large vs small p
In-out q	Biases local/outward movement	BFS/DFS tradeoff	weights 1 or $1/q$	Correct parameter direction
BFS-like	Samples local neighborhood	Homophily/community	Same social group	Usually $q > 1$
DFS-like	Moves outward	Structural equivalence	Hubs in separate groups	Usually $q < 1$
Alias sampling	Precomputed categorical sampler	Fast transitions	Per directed edge table	Memory tradeoff

5. Algorithm / Working Process

1. For each possible previous-current directed edge (t, v) , compute unnormalized weights for every x in $N(v)$.
2. Normalize or build alias tables.
3. Start multiple walks from each node; choose the first step normally.
4. For later steps, sample using the p, q conditional distribution.
5. Create context pairs and optimize Skip-gram negative-sampling loss.
6. Evaluate learned node vectors on the target task.

6. Mathematical Foundation

For edge weight w_{vx} , transition weight after traversing $t \rightarrow v$ is:

```

pi_vx = alpha_pq(t,x) * w_vx
alpha_pq(t,x) = 1/p    if distance(t,x)=0
                1      if distance(t,x)=1
                1/q    if distance(t,x)=2
P(x | v,t) = pi_vx / sum_(z in N(v)) pi_vz

```

Small p encourages immediate return. $q > 1$ downweights outward nodes and is more local/BFS-like; $q < 1$ favors outward exploration and is more DFS-like. The Skip-gram loss is the same as DeepWalk.

7. Practical Implementation

```

import random

graph = {0:{1:1,2:1}, 1:{0:1,2:1}, 2:{0:1,1:1,3:1}, 3:{2:1,4:1}, 4:{3:1}}

def node2vec_walk(start, length, p=1.0, q=1.0):
    walk = [start]
    while len(walk) < length and graph[walk[-1]]:
        current = walk[-1]
        candidates = list(graph[current])
        if len(walk) == 1:
            walk.append(random.choice(candidates)); continue
        previous = walk[-2]
        weights = []
        for nxt in candidates:
            if nxt == previous:
                bias = 1 / p

```

```

        elif nxt in graph[previous]:
            bias = 1
        else:
            bias = 1 / q
        weights.append(graph[current][nxt] * bias)
        walk.append(random.choices(candidates, weights=weights, k=1)[0])
    return walk

random.seed(2)
local_walks = [node2vec_walk(0, 8, p=1, q=4) for _ in range(3)]
outward_walks = [node2vec_walk(0, 8, p=1, q=.25) for _ in range(3)]
print("local:", local_walks)
print("outward:", outward_walks)

```

Feed these walks into the DeepWalk Skip-gram training code; only the sampler changes.

8. Code Explanation

The candidate `nxt` is categorized by its shortest possible distance from `previous`: return, adjacent, or outward. Edge weight multiplies the search bias. `random.choices` normalizes internally. This direct implementation is clear but recomputes membership and probabilities; production implementations preprocess sets and alias tables.

9. Training / Evaluation

Treat `p, q` as data- and task-dependent, not magic constants. Grid-search log-scale values such as `{0.25, 0.5, 1, 2, 4}` on validation performance. Tune walk/window/embedding parameters as in DeepWalk. Use identical edge splits, negative sets, probe models, and seeds when comparing. Analyze both community labels and structural-role tasks: a setting good for one can harm the other.

10. Complexity and Cost

Walk generation remains $O(nrL)$ samples, but second-order probability computation is costlier. Naive sampling scans $\deg(v)$ each step. Alias preprocessing gives near- $O(1)$ sampling but can require tables proportional to transitions across directed edges, potentially large around hubs. Skip-gram time and $O(nd)$ embedding memory match DeepWalk.

11. Common Use Cases

- Social recommendation and community classification
- Role discovery in communication/computer networks
- Citation and collaboration similarity
- Structure-based fraud/anomaly features
- Link prediction on fixed graphs
- Graph-based candidate retrieval

12. Common Mistakes

- Reversing `q`: `q > 1` is more inward/local; `q < 1` more outward
- Describing `p` as the probability of return rather than an inverse weight
- Calling it fully DFS/BFS—it is a stochastic interpolation
- Using directed/weighted graphs without adapting distance and transitions
- Tuning on test labels
- Assuming `p, q` guarantee structural-role embeddings
- Ignoring DeepWalk (`p=q=1`) as a baseline

13. Edge Cases / Limitations

The `p, q` interpretation is clearest for unweighted undirected graphs. Directed graphs may make distances asymmetric; leaves force returns regardless of bias. Large hubs make transition preprocessing expensive. Like DeepWalk, Node2Vec ignores attributes and is transductive. Global roles may still require specialized structural descriptors.

14. Variations

Variation	Change	When to use	Importance
Weighted Node2Vec	Multiplies bias by edge weight	Strength-aware networks	Projects
Directed Node2Vec	Respects outgoing transitions	Follows/citations	Practical
Node2Vec+	Refines noisy weighted-edge bias	Weighted graphs	Research
Attributed walks	Bias uses feature similarity	Structure + attributes	Advanced projects
Metapath2Vec	Type/schema-constrained contexts	Heterogeneous graphs	Core extension

15. Related Topics

$p=q=1$ reduces Node2Vec to DeepWalk-style uniform walking. LINE preserves explicit first/second-order adjacency without walk sequences. Struc2Vec targets structural equivalence more directly. GraphSAGE is inductive and feature-aware. Personalized PageRank provides a restart-based local diffusion that resembles controlled exploration but has no Skip-gram objective.

16. Interview Questions

- What does Node2Vec change from DeepWalk?** It adds second-order transition bias using previous node and p, q .
- Meaning of p ?** Return parameter; transition weight back to the previous node is $1/p$.
- Meaning of q ?** In-out parameter; outward candidate weight is $1/q$.
- What does small p do?** Encourages immediate backtracking.
- What does large q do?** Discourages outward moves, yielding more local/BFS-like walks.
- When is it DFS-like?** Generally $q < 1$, which increases outward relative weight.
- Why second-order?** The previous node is needed to classify candidates as return, nearby, or outward.
- What are homophily and structural equivalence?** Similarity through connection/shared community versus similarity of graph role despite distance.
- Is Node2Vec inductive?** Standard Node2Vec is not; it learns known-node embeddings.
- How choose p, q ?** Validate for the downstream task; graph structure alone does not determine the optimum.
- Why alias sampling?** It converts repeated categorical transitions into constant-time draws after preprocessing.
- What happens at $p=q=1$?** All unweighted neighbor candidates are uniform, like DeepWalk.

17. Practice Tasks

- Verify empirical transition frequencies against the p, q formula.
- Compare community classification for $q = .25, 1, 4$.
- Construct two distant star hubs and test role similarity.
- Find a directed-edge bug caused by symmetric neighbor lookup.
- Add edge weights and alias sampling to the toy implementation.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Node2Vec Explorer	Visualizes how p, q alter walks and embeddings	Streamlit, NetworkX, PyTorch	Karate/Les Misérables	Strong interview demo
Fraud Role Finder	Finds wallets with similar transaction roles	Node2Vec, XGBoost	Elliptic	Hybrid graph/tabular ML
Research Collaboration Recommender	Suggests collaborators from biased walks	PyG/Gensim, FAISS	DBLP	Retrieval + tuning

19. Quick Revision

- Key idea:** bias walks between local community and outward exploration.

- **Main formula:** weights $1/p$, 1 , $1/q$ for distance $0, 1, 2$.
- **When to use:** fixed structure-only graph needing tunable proximity.
- **Metrics:** downstream F1, MRR/Recall@K, NMI.
- **Traps:** reversing q ; claiming guaranteed BFS/DFS or induction.
- **Interview one-liner:** "Node2Vec is DeepWalk with a second-order p, q walk that controls return and outward exploration."

20. Final Cheat Sheet

Item	Summary
Input/output	Graph -> embedding for each known node
Main steps	Biased walks, contexts, Skip-gram
Hyperparameters	p, q, d , walks, length, window, negatives
Metrics	Probe F1, MRR/Recall@K, clustering NMI
Pros/cons	Flexible topology sampling / costly tuning, no cold start
Best uses	Community/role-aware transductive embeddings

Graph Neural Networks

1. Overview

Graph Neural Networks (GNNs) are neural models designed for graph-structured data. Most use **message passing**: nodes repeatedly receive information from neighbors, aggregate it with a permutation-invariant operation, and update their hidden state. GNNs learn node-, edge-, or graph-level outputs end-to-end for molecules, recommendations, fraud, traffic, physical simulation, and knowledge systems.

2. Intuition

Imagine every node as a person who summarizes messages from immediate contacts. After one round, a person knows one-hop context; after two, information has traveled two hops. Everyone uses shared rules, so the model works regardless of node ordering or graph size.

3. Prerequisites

- Graph representations, neighborhoods, degree, and sparse tensors
- MLPs, backpropagation, activations, regularization
- Linear algebra and permutation invariance/equivariance
- Node/link/graph prediction tasks and evaluation splits

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Message	Information sent along an edge	Can use sender/receiver/edge data	$M(h_i, h_j, e_{ji})$	Direction convention
Aggregation	Combines neighbor messages	Must ignore neighbor ordering	sum/mean/max	Expressiveness of sum
Update	Merges aggregate with current state	Retains self-information	MLP/GRU	Residual connections
Readout	Pools node states	Produces graph prediction	sum + MLP	Permutation invariance

Subtopic	Meaning	Why it matters	Example	Interview angle
Receptive field	Nodes reachable in L layers	Controls context	L hops	Neighborhood explosion
Oversmoothing	Deep node states become similar	Limits depth	indistinguishable nodes	Remedies
Oversquashing	Many distant signals compressed	Hurts long-range tasks	bottleneck cut	Rewiring/attention not cure-all
Expressiveness	Graph distinctions model can make	Sets theoretical ceiling	1-WL relation	MPNN limitations

5. Algorithm / Working Process

For each layer l :

1. **Input:** graph structure, node state $h_i^{(l)}$, optional edge feature e_{ji} .
2. Compute messages from neighbor j to receiver i .
3. Aggregate the multiset of incoming messages.
4. Update node i , optionally normalize, activate, drop out, or add a residual.
5. Repeat for L layers.
6. **Output:** node states directly, pairwise decoded edge scores, or a pooled graph vector.
7. Train end-to-end with task loss; inference is the same forward propagation without gradient updates.

6. Mathematical Foundation

The Message Passing Neural Network framework is:

$$\begin{aligned}
 m_i^{(l)} &= \text{AGG}_{\{j \in N(i)\}} M_l(h_i^{(l)}, h_j^{(l)}, e_{ji}) \\
 h_i^{(l+1)} &= U_l(h_i^{(l)}, m_i^{(l)}) \\
 y_G &= R(\{h_i^{(L)} \mid i \in V\})
 \end{aligned}$$

AGG and graph readout **R** must be permutation invariant (sum/mean/max/attention over a set). Node mappings are permutation equivariant: permuting inputs permutes outputs. Training minimizes a task loss, for example node cross-entropy, link BCE, or graph MSE. Under common assumptions, standard MPNNs are at most as discriminative as the 1-dimensional Weisfeiler-Lehman test.

7. Practical Implementation

```

import torch
from torch import nn
import torch.nn.functional as F

class MessagePassingLayer(nn.Module):
    def __init__(self, width, edge_dim):
        super().__init__()
        self.message = nn.Linear(2 * width + edge_dim, width)
        self.update = nn.Linear(2 * width, width)

    def forward(self, h, edge_index, edge_attr):
        src, dst = edge_index
        messages = F.relu(self.message(torch.cat([h[dst], h[src], edge_attr], 1)))
        total = torch.zeros_like(h).index_add_(0, dst, messages)
        count = torch.bincount(dst, minlength=len(h)).clamp(min=1).unsqueeze(1)
        return F.relu(self.update(torch.cat([h, total / count], 1)))

class GraphRegressor(nn.Module):
    def __init__(self, in_dim=3, width=16, edge_dim=1):

```

```

    super().__init__()
    self.embed = nn.Linear(in_dim, width)
    self.mp = MessagePassingLayer(width, edge_dim)
    self.out = nn.Linear(width, 1)
    def forward(self, x, edge_index, edge_attr, batch):
        h = self.mp(F.relu(self.embed(x)), edge_index, edge_attr)
        pooled = torch.zeros(batch.max().item()+1, h.size(1)).index_add_(0, batch, h)
        return self.out(pooled).squeeze(1)

# Two graphs in one disconnected batch.
x = torch.randn(5, 3); batch = torch.tensor([0,0,0,1,1])
edge_index = torch.tensor([[0,1,1,2,3,4], [1,0,2,1,4,3]])
edge_attr = torch.ones(edge_index.size(1), 1)
assert GraphRegressor()(x, edge_index, edge_attr, batch).shape == (2,)

```

8. Code Explanation

Each directed edge constructs a learned message from receiver state, sender state, and edge feature. `index_add_` implements order-independent sum aggregation; division produces a mean. `batch` maps nodes to graphs, and another sum yields one vector per graph. Real workloads normally use PyTorch Geometric or DGL for optimized scatter, batching, and sampling.

9. Training / Evaluation

Split by the true unit of generalization: nodes for transductive classification, whole graphs/scaffolds for molecules, timestamps for events, or entities for cold start. Normalize continuous features using training statistics. Batch graphs or sampled neighborhoods, monitor train/validation loss, and use early stopping. Tune layers, width, aggregation, dropout, normalization, residuals, learning rate, batch size, and sampling. Compare non-graph and simple structural baselines. Evaluate by graph size, degree, homophily, and unseen structure—not only a global average.

10. Complexity and Cost

A dense linear message-passing layer is roughly $O(md + nd^2)$ with width d ; memory is $O(m+nd)$ plus activations for backpropagation. Full-batch training is bounded by the entire graph. Neighbor sampling lowers each batch but can cause exponential fanout k^L , duplicated work, and sampling variance. Graph-level mini-batches are natural for many small graphs. GPUs help sparse aggregation, though irregular memory access can be the bottleneck.

11. Common Use Cases

- Molecular property and protein prediction
- Fraud, risk, and cybersecurity graphs
- Recommenders and matching systems
- Traffic forecasting and routing signals
- Knowledge-graph reasoning
- Mesh/particle physical simulation
- Program, dependency, and scene-graph analysis

12. Common Mistakes

- Wrong source/destination convention or missing reverse edges
- Non-invariant aggregation by concatenating arbitrary neighbor order
- Target-edge or future-edge leakage into message passing
- Omitting self-information or normalization unintentionally
- Excess depth without diagnosing smoothing/squashing
- Ignoring edge attributes/types
- Comparing with no MLP/heuristic baseline
- Mixing nodes from different graphs during readout
- Using random molecular splits when scaffold generalization is claimed

13. Edge Cases / Limitations

Standard message passing struggles with long-range dependencies, heterophily, graph symmetries indistinguishable to 1-WL, dynamic structure, and noisy/adversarial edges. Large graphs create memory and sampling challenges. Explanation and calibration are difficult, and performance may largely reflect data construction rather than architecture. Graph positional information is not automatic.

14. Variations

Variation	Change	When to use	Importance
GCN	Symmetric normalized neighbor sum	Homophilous full graph	Essential
GraphSAGE	Sample + inductive aggregation	Large/unseen nodes	Essential
GAT	Learned neighbor weights	Unequal neighbor relevance	Essential
GIN	Sum + MLP, strong 1-WL expressiveness	Graph classification	Interviews/research
R-GCN	Relation-specific transforms	Typed edges/KGs	Industry/research
Temporal GNN	Memory/time-aware messages	Event streams	Advanced

15. Related Topics

CNNs are message passing on a regular grid with fixed ordered neighborhoods; Transformers are attention-based set/sequence models often adapted to graphs. Graph embeddings may be fixed lookup tables, whereas GNNs are learnable encoders. GCN, GraphSAGE, and GAT differ primarily in aggregation and scaling. Geometric deep learning supplies the symmetry-first umbrella.

16. Interview Questions

- What is message passing?** Repeatedly compute edge messages, aggregate by receiver, and update node states.
- Why must aggregation be invariant?** Neighbor ordering is arbitrary and has no graph semantics.
- Why shared parameters?** They provide permutation equivariance and generalize across nodes/graph sizes.
- What does two layers capture?** At most two-hop information, assuming conventional local edges.
- Oversmoothing vs oversquashing?** Smoothing makes states alike; squashing compresses too much distant information through bottlenecks.
- How produce graph output?** Apply invariant readout such as sum/mean/max or learned set pooling.
- Why can sum be more expressive than mean?** Sum can distinguish multisets with equal averages and retain size/count.
- What is the 1-WL limitation?** Standard MPNNs cannot distinguish some non-isomorphic graphs that color refinement also cannot.
- How use edge features?** Include them in the message function or attention score.
- How scale to a billion edges?** Sampling, partitioning, distributed storage/training, cached features, and simplified propagation.
- Are attention weights explanations?** Not automatically; they are model-internal coefficients and need faithfulness checks.
- What is a valid baseline?** Feature-only MLP plus relevant graph heuristic/shallow method.

17. Practice Tasks

- Implement sum, mean, and max aggregation and test permutation equivariance.
- Train a graph classifier on MUTAG or a molecular dataset.
- Plot smoothing as average pairwise cosine similarity across depth.
- Debug a batched readout that mixes graph IDs.
- Add edge features and residual connections to the example.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Molecular Property Lab	Predicts solubility with scaffold evaluation	PyG, RDKit, MLflow	ESOL/MoleculeNet	Research-quality splitting
Transaction Risk GNN	Scores accounts and transfers over time	PyG, XGBoost, FastAPI	Elliptic	Production fraud story
Mesh Physics Emulator	Predicts node motion on meshes	PyTorch, PyG	MeshGraphNets-style synthetic	Geometric DL depth

19. Quick Revision

- **Key idea:** shared local message, invariant aggregate, node update.
- **Main formula:** $h_i' = U(h_i, \text{AGG}_j M(h_i, h_j, e_{ji}))$.
- **When to use:** relationships affect node/edge/graph target.
- **Metrics:** task-specific plus slice/generalization checks.
- **Traps:** leakage, direction, batching, smoothing/squashing.
- **Interview one-liner:** "A GNN is a permutation-equivariant neural computation over relational neighborhoods."

20. Final Cheat Sheet

Item	Summary
Input/output	Graph features -> node, edge, or graph predictions
Main steps	Message, aggregate, update, readout, task head
Hyperparameters	Layers, width, aggregator, dropout, sampling
Metrics	Depends on task; always include robust split/slices
Pros/cons	Relational inductive bias / scaling and long-range limits
Best uses	Molecules, networks, recommendations, physics

GCN

1. Overview

The Graph Convolutional Network (GCN) of Kipf and Welling performs a normalized linear aggregation of each node and its neighbors, followed by a nonlinearity. It is a canonical semi-supervised node-classification model and a useful baseline for homophilous graphs. Its "convolution" is an efficient first-order approximation motivated by spectral graph filters.

2. Intuition

Each node averages transformed features from itself and nearby nodes. Degree normalization prevents a popular node from blindly producing a much larger value and balances messages between low- and high-degree endpoints. Repeating the operation smooths features over the graph.

3. Prerequisites

- Adjacency/degree matrices and graph Laplacian
- Matrix multiplication, eigenvectors, normalization
- GNN message passing, cross-entropy, PyTorch
- Homophily, sparse tensors, train masks

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Self-loop	$\tilde{A} = A + I$	Retains own features	node messages itself	Why add before degree?
Symmetric normalization	$D^{-1/2} \tilde{A} D^{-1/2}$	Stabilizes hub scaling	degree-weighted mean	Versus random-walk norm
Propagation	Multiply normalized adjacency by states	Neighborhood smoothing	$\hat{A} H$	Order with W
Shared transform	W applies to every node	Learnable feature mixing	linear layer	Parameter count independent of n
Full batch	One forward over graph	Simple but memory-heavy	Cora	Scaling limit
Oversmoothing	Repeated low-pass filtering	Limits layers	class boundaries blur	Why 2 layers common?

5. Algorithm / Working Process

1. Input adjacency A , features X , labels, and masks.
2. Add self-loops: $\tilde{A} = A + I$.
3. Compute $\tilde{D}$ from $\tilde{A}$ and normalized $\hat{A}$.
4. For each layer, apply $H^{(l+1)} = \text{activation}(\hat{A} H^{(l)} W^{(l)})$.
5. Final layer yields node logits.
6. Train via masked loss; at inference run propagation and select target-node outputs.

6. Mathematical Foundation

```

A_tilde = A + I
D_tilde_ii = sum_j A_tilde_ij
A_hat = D_tilde^(-1/2) A_tilde D_tilde^(-1/2)
H^(l+1) = sigma(A_hat H^(l) W^(l)), H^(0)=X

```

Message from j to i has scale $1/\sqrt{d_i d_j}$. For node classification:

```

Z = softmax(A_hat ReLU(A_hat X W0) W1)
L = -sum_(i in train) log Z_(i,y_i)

```

Spectrally, the layer behaves as a learnable low-pass filter approximation, encouraging local smoothness. Weight decay and dropout regularize the small labeled regime.

7. Practical Implementation

```

import torch
from torch import nn
import torch.nn.functional as F

def normalized_adjacency(A):
    A = A + torch.eye(A.size(0), device=A.device)
    degree = A.sum(1)
    scale = degree.rsqrt()
    return scale[:, None] * A * scale[None, :]

class GCN(nn.Module):

```

```

def __init__(self, in_dim, hidden, classes):
    super().__init__()
    self.w1 = nn.Linear(in_dim, hidden, bias=False)
    self.w2 = nn.Linear(hidden, classes, bias=False)
def forward(self, x, A_hat):
    h = F.relu(A_hat @ self.w1(x))
    h = F.dropout(h, p=.5, training=self.training)
    return A_hat @ self.w2(h)

A = torch.tensor([[0.,1,1,0],[1,0,1,0],[1,1,0,1],[0,0,1,0]])
x = torch.eye(4); y = torch.tensor([0,0,0,1])
mask = torch.tensor([True,True,False,True])
A_hat = normalized_adjacency(A)
model = GCN(4, 8, 2); opt = torch.optim.Adam(model.parameters(), lr=.02)
for _ in range(100):
    model.train(); opt.zero_grad()
    loss = F.cross_entropy(model(x, A_hat)[mask], y[mask])
    loss.backward(); opt.step()
assert model(x, A_hat).shape == (4, 2)

```

8. Code Explanation

Self-loops are added before recomputing degrees. Outer multiplication by inverse square-root degrees implements symmetric normalization without explicitly forming diagonal matrices. Each layer first transforms features then propagates them; associativity makes this equivalent to $A_hat @ X @ W$. Dense A keeps the example readable; use sparse `GCNConv` in PyTorch Geometric for real graphs.

9. Training / Evaluation

Use training/validation/test masks and compute loss only on training nodes. Tune hidden width, dropout, learning rate, weight decay, depth, early-stopping patience, and normalization. Standard citation benchmarks are transductive; state that clearly. Report accuracy/macro-F1 across seeds and compare MLP and label-propagation baselines. For evolving graphs, use chronological splits and decide whether inference can access new edges.

10. Complexity and Cost

Sparse propagation costs $O(md)$ and feature transformation $O(nd_in\ d_out)$ per layer. Parameters cost $O(d_in\ d_out)$ independent of graph size; activations cost $O(ndL)$ during training. Dense adjacency incorrectly raises propagation to $O(n^2d)$. Full-batch GCN requires all features/edges; sampling or simplified precomputation is used at scale.

11. Common Use Cases

- Citation topic and document classification
- Homophilous social-node labeling
- Graph autoencoder encoders
- Molecular node/graph feature extraction
- Semi-supervised classification with few labels
- Baseline for new GNN research

12. Common Mistakes

- Forgetting self-loops or adding them twice
- Normalizing with degrees computed before self-loops
- Using dense adjacency on a large sparse graph
- Applying loss to all node labels
- Confusing symmetric normalization with simple mean
- Too many layers without residuals/normalization
- Expecting strong performance on heterophily
- Calling standard full-graph GCN inherently inductive

13. Edge Cases / Limitations

GCN smoothing is poorly matched to strongly heterophilous edges. Isolated nodes only use self-features. Deep stacks oversmooth; bottlenecks oversquash long-range signals. Static full-batch computation is awkward for rapidly changing or enormous graphs. Symmetric normalization assumes a simple treatment of direction, weights, and edge types.

14. Variations

Variation	What changes	When to use	Importance
ChebNet	Polynomial spectral filters	Wider spectral support	Research foundation
SGC	Precompute propagation, remove nonlinear layers	Fast baseline	Interviews/projects
APPNP	Personalized-PageRank propagation	Deeper reach, less smoothing	Advanced
GCNII	Initial/residual identity mappings	Deep GCNs	Research
R-GCN	Relation-specific weights	Typed/KG edges	Important

15. Related Topics

GraphSAGE uses explicit inductive aggregators and sampling; GAT replaces fixed degree weights with learned attention. SGC shows much GCN performance comes from feature propagation. Label propagation smooths labels rather than learned features. CNN convolution uses fixed grid locality, while GCN constructs locality from adjacency.

16. Interview Questions

- Write the GCN layer.** $H' = \sigma(D_{\tilde{}}^{-1/2} A_{\tilde{}} D_{\tilde{}}^{-1/2} H W)$.
- Why self-loops?** They preserve the node's own signal during aggregation.
- Why symmetric normalization?** It controls scale using both sender and receiver degree and yields a symmetric operator on undirected graphs.
- What is the receptive field of two layers?** Up to two graph hops.
- Why is GCN called spectral?** It is motivated as a localized approximation to Laplacian spectral filtering.
- Why is it a low-pass/smoothing operator?** Neighbor averaging suppresses high-frequency differences across edges.
- What causes oversmoothing?** Repeated propagation drives representations toward low-frequency/stationary subspaces.
- Can GCN use directed graphs?** Yes with a chosen directed normalization or symmetrization, but semantics must be explicit.
- Parameter and propagation complexity?** Parameters depend on feature widths; sparse propagation is linear in edges times width.
- GCN vs MLP?** GCN mixes neighboring nodes; MLP transforms each node independently.
- When does GCN fail?** Heterophily, long-range bottlenecks, noisy edges, or scale beyond full batch.
- How scale it?** Sparse operations, sampling/clusters, precomputed diffusion, or distributed training.

17. Practice Tasks

- Derive and calculate $A_{\hat{}}$ for a three-node path.
- Implement sparse GCN and compare it with the dense version.
- Train GCN vs MLP on Cora and report five-seed mean/std.
- Diagnose a bug where degrees exclude self-loops.
- Explore depth with residuals and an oversmoothing measure.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Citation GCN Reproduction	Reproduces baseline with rigorous seeds	PyG, MLflow	Cora/CiteSeer	Experimental discipline
Fraud Neighborhood Classifier	Combines transaction features and topology	PyG, FastAPI	Elliptic	End-to-end deployment

Project	What it does	Stack	Dataset	Resume value
GCN Interpretability Dashboard	Shows influential edges/features	PyG, Captum, Streamlit	PubMed	Explainability skills

19. Quick Revision

- **Key idea:** degree-normalized smoothing plus feature transform.
- **Main formula:** $H' = \sigma(D^{-1/2} A D^{-1/2} H W)$.
- **When to use:** homophilous attributed graphs and strong baseline needs.
- **Metrics:** task metric, seed variance, degree/homophily slices.
- **Traps:** self-loop normalization, dense A , excess depth.
- **Interview one-liner:** "GCN is a first-order normalized graph low-pass filter with learned channel mixing."

20. Final Cheat Sheet

Item	Summary
Input/output	$X, A \rightarrow$ contextual node states/logits
Main steps	Add loops, normalize, propagate, transform, activate
Hyperparameters	Layers, hidden width, LR, dropout, weight decay
Metrics	Accuracy/F1 or downstream task metric
Pros/cons	Simple/strong / full-batch, heterophily, smoothing
Best uses	Homophilous semi-supervised node tasks

GraphSAGE

1. Overview

GraphSAGE (Sample and Aggregate) is an inductive GNN framework. Instead of learning only an embedding per node ID, it learns functions that sample neighbors, aggregate their features, and combine them with the center node. It enables mini-batch training on large graphs and embeddings for unseen nodes when their features and neighborhoods are available.

2. Intuition

To describe a new customer, sample a manageable number of contacts, summarize them, and combine that summary with the customer's own profile. The same recipe learned on existing users applies to a newly arrived user; no dedicated lookup vector is required.

3. Prerequisites

- Message passing and node classification
- Mini-batches, stochastic optimization, sampling variance
- Set aggregation and permutation invariance
- Supervised and contrastive/negative-sampling losses

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Inductive encoder	Shared function of attributes/neighbors	Handles unseen nodes	new product	Conditions for induction
Neighbor sampling	Fixed fanout per layer	Controls memory	[15, 10]	Exponential fanout

Subtopic	Meaning	Why it matters	Example	Interview angle
Mean aggregator	Average neighbor states	Fast and invariant	neighbor mean	Relation to GCN
Pooling/LSTM aggregator	Learned nonlinear/sequence summary	More expressive/costly	max-pool MLP	LSTM order problem
Concatenation	Keeps self distinct from neighbor	Useful under heterophily	$[h_i]$	
Unsupervised loss	Nearby nodes positive; sampled nodes negative	Learns without labels	random-walk pairs	Transductive pairs, inductive encoder

5. Algorithm / Working Process

1. Select a mini-batch of target nodes.
2. Recursively sample a fixed number of neighbors for each layer.
3. Starting at the outermost sampled nodes, aggregate representations inward.
4. Concatenate or combine each node's current state with its neighbor aggregate.
5. Transform, activate, and optionally L2-normalize.
6. Apply task head and loss on seed nodes only.
7. For inference, sample or aggregate neighborhoods of new nodes with the same learned functions.

6. Mathematical Foundation

$$h_N(i)^k = \text{AGG}_k(\{h_j^{(k-1)}, j \in \text{sampled } N(i)\})$$

$$h_i^k = \text{sigma}(W^k [h_i^{(k-1)} \parallel h_N(i)^k])$$

$$h_i^k = h_i^k / ||h_i^k||_2 \quad (\text{optional})$$

For mean aggregation, $\text{AGG}(S) = |S|^{-1} \sum_{h \in S} h$. A commonly described unsupervised objective is:

$$-\log \text{sigmoid}(z_u^T z_v) - Q \mathbb{E}_{(v \sim P_v)} \log \text{sigmoid}(-z_u^T z_v)$$

where v co-occurs with u in short walks. Supervised training instead uses the relevant node/link/graph loss.

7. Practical Implementation

```
import torch
from torch import nn
import torch.nn.functional as F

class SAGEConv(nn.Module):
    def __init__(self, in_dim, out_dim):
        super().__init__()
        self.linear = nn.Linear(2 * in_dim, out_dim)
    def forward(self, x, edge_index):
        src, dst = edge_index
        summed = torch.zeros_like(x).index_add_(0, dst, x[src])
        count = torch.bincount(dst, minlength=len(x)).clamp(min=1).unsqueeze(1)
        return self.linear(torch.cat([x, summed / count], dim=1))

class GraphSAGE(nn.Module):
    def __init__(self, in_dim, hidden, classes):
        super().__init__()
        self.sage1 = SAGEConv(in_dim, hidden)
        self.sage2 = SAGEConv(hidden, classes)
```

```
def forward(self, x, edge_index):
    x = F.relu(self.sage1(x, edge_index))
    x = F.dropout(x, .3, self.training)
    return self.sage2(x, edge_index)

x = torch.randn(5, 4)
edge_index = torch.tensor([[0,1,1,2,2,3,3,4], [1,0,2,1,3,2,4,3]])
assert GraphSAGE(4, 8, 3)(x, edge_index).shape == (5, 3)
# At scale replace full edge_index with sampled blocks/NeighborLoader in PyG.
```

8. Code Explanation

Mean aggregation is implemented with sum scatter and degree division. Concatenation keeps center and neighbor channels separate, a defining practical distinction from a simple normalized GCN update. Parameters depend on feature dimensions, not node IDs. This example uses all neighbors so the layer is visible; production GraphSAGE obtains a sampled computation graph per seed batch.

9. Training / Evaluation

For a real inductive test, hold out nodes and their incident training access according to the deployment contract, or train on some graphs and test on entirely different graphs. Tune per-layer fanout, layers, width, aggregator, batch size, dropout, LR, and whether inference uses sampling or all neighbors. Sampling must start from seed nodes, and loss must apply only to seeds. Report speed/memory and accuracy variance alongside task metrics; compare full-neighbor inference with sampled inference.

10. Complexity and Cost

With batch size B , fanouts $s_1 \dots s_L$, sampled nodes can grow near $B \prod_{l=1}^L s_l$; repeated nodes reduce the actual count. Per-batch compute is proportional to sampled edges and dense transforms. Sampling bounds memory but adds CPU/data-loader cost, variance, and duplicate neighborhood computation. Layer-wise inference over all nodes/edges avoids exponential sampling but needs graph-wide passes.

11. Common Use Cases

- New-user/product classification and recommendation
- Large social and transaction graphs
- Dynamic catalogs with node attributes
- Web-scale node embeddings
- Pinterest-style content recommendation
- Inductive inference across related graphs

12. Common Mistakes

- Calling the model inductive while using only learned node IDs
- Holding out labels but leaving an unrealistic inference graph
- Applying loss to sampled context nodes instead of seeds
- Confusing number of sampled neighbors with total receptive-field size
- Sampling before defining temporal/data boundaries
- Using LSTM aggregation with arbitrary unstable neighbor ordering
- Ignoring full-neighbor vs sampled inference mismatch
- Assuming mean aggregation preserves neighbor count

13. Edge Cases / Limitations

Nodes without useful features or neighbors remain hard. Popular nodes are poorly approximated by small uniform samples; rare important neighbors may be missed. Multi-layer fanout still explodes. Sampling overhead and staleness can dominate distributed training. Mean aggregation loses multiset cardinality and may blur heterophilous neighborhoods.

14. Variations

Variation	What changes	When to use	Importance
Mean SAGE	Neighbor average	Default scalable baseline	Essential
Pooling SAGE	MLP per neighbor + max	Nonlinear set patterns	Projects
LSTM SAGE	LSTM over shuffled neighbors	Historical experiments	Interview caveat
PinSAGE	Importance sampling + random-walk neighborhoods	Web recommendation	Industry
Temporal sampling	Only past neighbors	Event graphs	High practical value

15. Related Topics

GCN uses a fixed symmetrically normalized sum and was introduced mainly full-batch/transductive; GraphSAGE explicitly samples and separates self from neighbor aggregation. GAT learns weights instead of uniform means. DeepWalk/Node2Vec use ID tables and fail cold start, while GraphSAGE can consume features. Cluster-GCN and GraphSAINT offer alternative scalable sampling units.

16. Interview Questions

1. **Why is GraphSAGE inductive?** It learns shared aggregation functions over features rather than only per-node vectors.
2. **What must a new node have?** Compatible features and an accessible neighborhood, or at least features for self-only inference.
3. **What does Sample and Aggregate mean?** Sample bounded neighborhoods, then summarize them layer by layer.
4. **Why concatenate self and neighbors?** It preserves their roles and lets the transform weight them differently.
5. **GraphSAGE vs GCN?** SAGE is explicitly inductive/sampled with configurable aggregators; GCN uses symmetric degree-normalized propagation.
6. **What is fanout?** Maximum sampled neighbors per node at a layer.
7. **Why does fanout grow?** Each sampled neighbor requires its own sampled predecessors for deeper layers.
8. **How handle hubs?** Importance/weighted sampling, larger fanout, caching, or full-neighbor layer-wise inference.
9. **Is mean unbiased?** Uniform sample mean can estimate full mean unbiasedly, but nonlinear multilayer outputs can still be biased/variable.
10. **What is PinSAGE?** A recommender adaptation using importance sampling from random walks and scalable convolution.
11. **Can SAGE be transductive?** Yes; inductive capability does not prevent training/evaluating on one fixed graph.
12. **How evaluate induction?** Hold out nodes/graphs/time consistently and forbid unavailable structure/features.

17. Practice Tasks

- Implement two-hop neighbor sampling and inspect batch expansion.
- Compare sampled and full-neighbor GraphSAGE on Reddit/Cora.
- Evaluate truly unseen nodes rather than only unseen labels.
- Debug loss accidentally computed over all sampled nodes.
- Add importance sampling and measure hub-node performance.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
New Product Recommender	Embeds unseen items from content and co-clicks	PyG, FAISS, FastAPI	Amazon Products	Cold-start + serving
Scalable Reddit Classifier	Benchmarks fanout/throughput/quality	PyG NeighborLoader, MLflow	Reddit	Systems-aware Graph ML
Inductive Fraud Scorer	Scores newly created accounts temporally	PyTorch, PyG	Elliptic/synthetic	Correct deployment protocol

19. Quick Revision

- **Key idea:** learned feature aggregator plus bounded neighbor sampling.
- **Main formula:** $h_i = \sigma(W[h_i \parallel \text{AGG}(N(i))])$.
- **When to use:** unseen nodes and/or large graphs.
- **Metrics:** task quality plus throughput, memory, seed variance.
- **Traps:** fake inductive split, fanout explosion, seed/context loss mix-up.
- **Interview one-liner:** "GraphSAGE trades exact full neighborhoods for learned inductive aggregation over sampled features."

20. Final Cheat Sheet

Item	Summary
Input/output	Features + sampled graph -> node embeddings/logits
Main steps	Sample recursively, aggregate inward, combine self, predict
Hyperparameters	Fanouts, aggregator, layers, width, batch size
Metrics	F1/AUC plus latency, throughput, memory
Pros/cons	Inductive/scalable / sampling bias, fanout, overhead
Best uses	Large dynamic attributed graphs and cold start

GAT

1. Overview

Graph Attention Networks (GATs) learn a different importance coefficient for each edge during message passing. Instead of weighting neighbors only by degree or uniformly, attention conditions on source and target representations. Multi-head attention stabilizes learning and provides multiple relational subspaces. GAT is useful when neighbor relevance varies, though attention is not automatically a faithful explanation.

2. Intuition

When asking colleagues for advice, you do not average everyone equally: relevance depends on both your question and each colleague. GAT learns this local weighting, but only among actual graph neighbors, so its attention is sparse compared with a standard global Transformer.

3. Prerequisites

- GNN message passing and softmax
- Attention, logits, LeakyReLU, multi-head mechanisms
- Sparse neighborhood operations and numerical stability
- Node classification and graph split protocols

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Attention logit	Learned compatibility on an edge	Scores relevance	$a^T [W h_i]$	
Neighborhood softmax	Normalizes over incoming neighbors	Comparable local weights	$\sum \alpha = 1$	Direction of softmax
Multi-head	Several independent attentions	Stability/subspaces	8 heads	Concat vs average
Masked attention	Only graph edges considered	Retains sparse inductive bias	$j \in N(i)$	Unlike Transformer

Subtopic	Meaning	Why it matters	Example	Interview angle
Attention dropout	Drops coefficients/features	Regularizes sparse focus	coefficient dropout	Train vs eval
Edge-aware attention	Adds relation/edge attributes	Models typed/weighted edges	bond feature	Vanilla limitation

5. Algorithm / Working Process

1. Linearly project node states: $z_i = W h_i$.
2. For every edge $j \rightarrow i$, compute compatibility logit from z_i, z_j (and optionally edge features).
3. Apply LeakyReLU and softmax across i 's incoming neighbors.
4. Weight and sum source messages by coefficients.
5. Apply activation; concatenate hidden-layer heads or average final heads.
6. Train with the task loss; inference recomputes attention for the current neighborhood.

6. Mathematical Foundation

Original additive GAT:

```
e_ij = LeakyReLU(a^T [W h_i || W h_j])
alpha_ij = exp(e_ij) / sum_(k in N(i)) exp(e_ik)
h_i' = sigma(sum_(j in N(i)) alpha_ij W h_j)
```

For K heads, hidden outputs are often concatenated: $h_i' = ||_{k=1}^K h_i'^k$; final heads may be averaged. Attention costs scale with edges rather than all node pairs. Cross-entropy/BCE/MSE depends on the downstream task, not GAT itself.

7. Practical Implementation

```
import torch
from torch import nn
import torch.nn.functional as F

class DenseGATHead(nn.Module):
    def __init__(self, in_dim, out_dim):
        super().__init__()
        self.W = nn.Linear(in_dim, out_dim, bias=False)
        self.a = nn.Parameter(torch.empty(2 * out_dim))
        nn.init.xavier_uniform_(self.a.view(1, -1))
    def forward(self, x, adjacency):
        z = self.W(x)
        n = len(x)
        zi = z[:, None, :].expand(n, n, -1)
        zj = z[None, :, :].expand(n, n, -1)
        logits = F.leaky_relu(torch.cat([zi, zj], -1) @ self.a, .2)
        mask = adjacency.bool() | torch.eye(n, dtype=torch.bool)
        alpha = logits.masked_fill(~mask, float("-inf")).softmax(dim=1)
        return alpha @ z, alpha

x = torch.randn(4, 5)
A = torch.tensor([[0,1,1,0],[1,0,1,0],[1,1,0,1],[0,0,1,0]])
head = DenseGATHead(5, 3)
h, attention = head(x, A)
assert h.shape == (4, 3)
assert torch.allclose(attention.sum(1), torch.ones(4))
```

8. Code Explanation

Every receiver-source pair is concatenated and scored, then non-edges are masked to negative infinity before row-wise softmax. Self-loops guarantee a valid row and retain self-signal. $\alpha @ z$ aggregates source projections. The implementation allocates $O(n^2)$ tensors only for clarity; use sparse `GATConv`, which evaluates logits only on edges.

9. Training / Evaluation

Use task-correct graph splits and compare with GCN/GraphSAGE at matched parameter counts. Tune heads, per-head width, layers, feature/attention dropout, negative slope, residuals, LR, and weight decay. Check attention entropy, degree-sliced metrics, seed variance, and compute cost. If presenting attention as an explanation, perform edge-removal/counterfactual tests and compare with gradient or perturbation attribution.

10. Complexity and Cost

Sparse projection costs $O(nd_{in}d)$ and attention/message operations about $O(md)$ per head. Coefficients require $O(mK)$ memory, which can be material on large graphs. Multi-head concatenation increases hidden width and subsequent transform cost. Unlike full self-attention's $O(n^2)$, graph masking is edge-linear, but hubs still create expensive softmax groups.

11. Common Use Cases

- Node classification with variable neighbor relevance
- Molecular graphs with edge-aware extensions
- Fraud networks with noisy connections
- Recommenders and user-item interactions
- Traffic/sensor graphs
- Heterogeneous relation weighting with adapted layers

12. Common Mistakes

- Applying softmax globally instead of within each receiver's neighbors
- Reversing source and destination grouping
- Forgetting self-loops, leaving isolated rows invalid
- Dense $n \times n$ attention on a sparse graph
- Comparing more-head GAT with a much smaller baseline
- Interpreting coefficients as causal explanations
- Ignoring edge features/types in a relation-rich graph
- Assuming learned weights solve oversquashing

13. Edge Cases / Limitations

Many attention heads increase memory and training variance. Neighbor softmax forces competition: adding/removing one neighbor changes all coefficients. Vanilla GAT's original scoring can have limited query-dependent ranking behavior, motivating GATv2. Low-information features yield weak attention; noisy hubs remain costly; attention cannot recover relationships absent from the graph.

14. Variations

Variation	What changes	When to use	Importance
GATv2	Reorders scoring transform for dynamic attention	More expressive pair ranking	Important interview follow-up
Edge-aware GAT	Edge features enter score/message	Molecules/typed links	Practical
Graph Transformer	Dot-product global/sparse attention + positional encodings	Long-range dependencies	Research/core modern

Variation	What changes	When to use	Importance
Heterogeneous attention	Type-specific projections/attention	Multi-relation graphs	Advanced
SuperGAT	Attention-supervision auxiliary objective	Improve edge discrimination	Research

15. Related Topics

GCN uses fixed degree-derived coefficients; GAT learns coefficients. GraphSAGE mean gives equal sampled-neighbor weights. Transformer attention commonly uses query-key dot products and global/token connectivity; GAT uses additive attention restricted to edges. GATv2 corrects a ranking limitation of the original scoring form.

16. Interview Questions

1. **What is GAT?** A message-passing GNN with learned normalized coefficients on graph edges.
2. **Write its coefficient.** $\alpha_{ij} = \text{softmax}_j(\text{LeakyReLU}(a^T [W_h i || W_h j]))$.
3. **Where is softmax applied?** Across source neighbors sending into the same receiver.
4. **Why multi-head?** Stabilizes optimization and learns different interaction subspaces.
5. **Concat vs average?** Hidden heads commonly concatenate; output heads often average to keep output size fixed.
6. **GAT vs Transformer?** GAT usually attends only over edges with additive scoring; Transformers often use global dot-product attention and positional encodings.
7. **Does GAT need degree normalization?** Softmax normalizes incoming weights, so not the GCN formula; degree still affects competition and cost.
8. **Is attention explainability?** It can be inspected but is not sufficient evidence of faithful feature/edge importance.
9. **Why self-loops?** To include self-state and make isolated-node normalization valid.
10. **What is GATv2's motivation?** More dynamic query-conditioned ranking of neighbors.
11. **Sparse complexity?** Roughly linear in edges times hidden width and heads, plus feature transforms.
12. **When can GCN beat GAT?** When fixed smoothing matches the graph, data are limited, or attention's extra variance/cost is unnecessary.

17. Practice Tasks

- Implement segment softmax over edge destinations.
- Verify output equivariance after permuting node IDs.
- Compare GCN/GAT at equal parameter budgets on Cora.
- Debug a GAT whose softmax is accidentally across all edges.
- Test whether high-attention edge removal truly changes predictions.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Attention Fraud Explorer	Scores accounts and audits influential neighbors	PyG, Captum, Streamlit	Elliptic	Model + explanation skepticism
Traffic Sensor GAT	Forecasts readings using learned sensor influence	PyTorch, PyG	METR-LA	Spatiotemporal modeling
Molecular Edge-GAT	Uses bond attributes for property prediction	PyG, RDKit	ESOL/BBBP	Scientific edge modeling

19. Quick Revision

- **Key idea:** learn normalized importance per graph edge.
- **Main formula:** $h_i' = \text{sigma}(\sum_j \alpha_{ij} W h_j)$.
- **When to use:** neighbor relevance varies and features support scoring.
- **Metrics:** task metric, cost, seed variance; faithfulness if explaining.

- **Traps:** wrong softmax group, dense attention, explanation claims.
- **Interview one-liner:** "GAT replaces fixed graph normalization with learned neighborhood attention while retaining sparse message passing."

20. Final Cheat Sheet

Item	Summary
Input/output	Graph/features -> attention-weighted node states
Main steps	Project, score edges, neighbor softmax, weighted sum
Hyperparameters	Heads, head width, dropout, layers, slope
Metrics	Task metric + compute/faithfulness diagnostics
Pros/cons	Adaptive neighbors / memory, variance, not explanation
Best uses	Noisy or unequal-relevance neighborhoods

Heterogeneous Graphs

1. Overview

A heterogeneous graph has multiple node types, edge/relation types, or both. Its schema may contain users, products, sellers, and categories connected by buys, views, sells, and belongs-to relations. Type-aware models preserve these distinct semantics instead of collapsing every entity and interaction into one homogeneous adjacency. They are central to recommendations, fraud, academic networks, healthcare, and enterprise data.

2. Intuition

An airport network containing passengers, flights, and airports cannot treat "passenger books flight" like "flight lands at airport." The endpoint types, feature spaces, and meaning differ. A heterogeneous GNN is like a multilingual message system: it translates each source type/relation into the receiver's representation space before combining messages.

3. Prerequisites

- Graph schemas, directed typed edges, and bipartite graphs
- GNN message passing, attention, and node/link tasks
- Dictionaries/tensor indexing for type-specific data
- Random walks and recommender/knowledge-graph basics

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Node type	Entity category with own semantics/features	Feature dimensions differ	user vs item	Type-specific encoders
Edge type	Typed relation, often directional	Interactions mean different things	buys vs views	Reverse relation handling
Canonical edge type	(source_type, relation, target_type)	Removes relation ambiguity	(user, buys, item)	Same name/different endpoints
Schema/metagraph	Allowed type-level connections	Constrains paths/models	author-paper-venue	Data validation
Metapath	Sequence of node/relation types	Defines composite semantics	author-paper-author	Manual bias vs learned model

Subtopic	Meaning	Why it matters	Example	Interview angle
Relation aggregation	Combines per-relation messages	Prevents semantic collapse	attention/sum	Parameter scaling
Type-specific target	Label exists for one/more types	Controls masks/heads	classify papers	Output head design

5. Algorithm / Working Process

1. Define a schema with node types and canonical directed edge types.
2. Build a feature matrix/encoder per node type and edge attributes per relation.
3. For each relation, transform source states and aggregate messages into its target type.
4. Combine multiple relation-specific aggregates at each target node.
5. Update target states using type-specific parameters.
6. Apply type-appropriate node heads, relation-aware link decoders, or graph readout.
7. Train with task loss and relation/type-aware sampling; infer using the same schema.

6. Mathematical Foundation

For target node i and relation set R_i :

$$m_i^r = \text{AGG}_{\{j \in N_r(i)\}} W_r h_j$$

$$h_i' = \text{sigma}(W_{\text{self}, \text{type}(i)} h_i + \text{COMBINE}_{\{r \in R_i\}} m_i^r)$$

R-GCN uses normalized relation sums:

$$h_i^{(l+1)} = \text{sigma}(\sum_{r \in R} \sum_{j \in N_i^r} (1/c_{i,r}) W_r^{(l)} h_j^{(l)} + W_{\theta}^{(l)} h_i^{(l)})$$

With many relations, basis decomposition reduces parameters: $W_r = \sum_b a_{(r,b)} V_b$. A metapath $A-P-A$ corresponds to multiplying compatible adjacency matrices, e.g. $A_{AP} A_{PA}$, producing author-author connectivity through papers.

7. Practical Implementation

```
import torch
from torch import nn
import torch.nn.functional as F

# users -> items through two relation types
x_user = torch.randn(3, 4)
x_item = torch.randn(4, 6)
buys = torch.tensor([[0,1,2], [1,2,1]]) # user IDs, item IDs
views = torch.tensor([[0,0,2,1], [0,1,3,3]])

class HeteroItemLayer(nn.Module):
    def __init__(self, hidden=8):
        super().__init__()
        self.user = nn.Linear(4, hidden)
        self.item = nn.Linear(6, hidden)
        self.rel = nn.ModuleDict({"buys": nn.Linear(hidden, hidden, bias=False),
                                   "views": nn.Linear(hidden, hidden, bias=False)})
        self.update = nn.Linear(3 * hidden, hidden)

    def aggregate(self, user_h, edges, relation, n_items):
        src, dst = edges
```

```

message = self.rel[relation](user_h[src])
total = message.new_zeros(n_items, message.size(1)).index_add_(0, dst, message)
count = torch.bincount(dst, minlength=n_items).clamp(min=1).unsqueeze(1)
return total / count

def forward(self, x_user, x_item, buys, views):
    u, i = F.relu(self.user(x_user)), F.relu(self.item(x_item))
    b = self.aggregate(u, buys, "buys", len(i))
    v = self.aggregate(u, views, "views", len(i))
    return F.relu(self.update(torch.cat([i, b, v], 1)))

item_embeddings = HeteroItemLayer()(x_user, x_item, buys, views)
assert item_embeddings.shape == (4, 8)

```

8. Code Explanation

Users and items first enter a shared hidden width through separate feature encoders. `buys` and `views` use separate relation transformations, then messages are averaged by destination item. Concatenating self, buy, and view summaries prevents premature semantic mixing. A full model also updates users through reverse relations and stacks layers; PyG `HeteroData/HeteroConv` or DGL heterographs automate that bookkeeping.

9. Training / Evaluation

Split by time and entity according to deployment; in recommendation, never let future views/purchases enter training neighborhoods. Ensure all target types/classes appear in validation without leaking identities. Use relation-balanced or type-aware samplers because frequent relations otherwise dominate. Tune type encoders, hidden width, relation parameter sharing, fanouts per relation, metapaths, attention heads, and loss weights. Evaluate each type/relation separately plus macro averages, cold-start slices, and rare-relation performance.

10. Complexity and Cost

Message passing is approximately $O(\sum_r m_r d + \text{parameter transforms})$. Independent $d \times d$ weights cost $O(|R|d^2)$ per layer; basis/block decomposition reduces this. Memory includes one feature/state store per node type and one edge index per canonical relation. Sampling is harder than homogeneous sampling because fanout and frequency vary across relations. GPUs help, but small relation partitions can underutilize kernels.

11. Common Use Cases

- User-item-category-seller recommendation
- Author-paper-venue academic mining
- Account-device-IP-transaction fraud graphs
- Patient-doctor-drug-diagnosis healthcare graphs
- Enterprise customer-contract-product graphs
- Multimodal scene/entity relations

12. Common Mistakes

- Collapsing relation types and destroying semantics
- Treating same numeric ID across types as the same node
- Forgetting reverse relations needed for message flow
- Using one feature encoder for incompatible feature spaces
- Sampling dominant relations only
- Leakage through metapaths containing target/future edges
- Creating separate full matrices for thousands of relations without sharing
- Reporting only aggregate metrics that hide rare-type failure

13. Edge Cases / Limitations

Rare relations receive weak training signal; new types/relations change the schema and often require retraining. Type-specific parameters overfit and grow rapidly. Metapaths encode expert bias and can miss useful patterns. Some graphs are “heterogeneous” only due to storage conventions; unnecessary typing increases complexity. Missing features and severe degree imbalance are common in enterprise graphs.

14. Variations

Variation	What changes	When to use	Importance
R-GCN	Relation-specific convolutions + sharing	Many typed edges/KGs	Essential
HAN	Attention over nodes and metapaths	Semantically meaningful metapaths	Interview/research
HGT	Type/relation-specific Transformer attention	Large rich schemas	Modern research/industry
Metapath2Vec	Typed random-walk embeddings	Shallow hetero baseline	Projects
Hetero GraphSAGE	Type-aware sampled aggregation	Large inductive graphs	Industry

15. Related Topics

A homogeneous graph has one node/edge type; a bipartite graph has two node sets and usually one interaction family; a heterogeneous graph generalizes both. Knowledge graphs are heterogeneous multi-relational graphs centered on factual triples. Metapaths resemble typed relational joins. HGT adapts Transformer attention, while R-GCN adapts GCN-style message passing.

16. Interview Questions

1. **What makes a graph heterogeneous?** Multiple node types, edge types, or distinct semantic feature spaces.
2. **What is a canonical edge type?** (source type, relation type, destination type).
3. **Why not merge all relations?** Their meanings and predictive effects can conflict.
4. **What is a metapath?** A schema-level sequence of types/relations defining a composite relation.
5. **Example metapath?** Author-Paper-Author represents co-authorship.
6. **How pass messages across different feature dimensions?** Type-specific input projections map them to compatible hidden spaces.
7. **How control relation parameter count?** Basis/block decomposition, shared backbones, or low-rank adapters.
8. **Why add reverse relations?** Directed messages otherwise flow only along recorded direction; targets may never inform sources.
9. **How sample hetero graphs?** Set fanouts and sampling distributions per canonical relation/type.
10. **Heterogeneous graph vs KG?** A KG is usually a typed factual triple graph; heterogeneity is the broader data/model category.
11. **How evaluate cold start?** Hold out complete entities/types as appropriate and use only available attributes/relations.
12. **Biggest leakage risk?** Future/target relations reappearing through reverse edges, features, or metapaths.

17. Practice Tasks

- Build a **user-item-category** typed graph with canonical relations.
- Implement R-GCN basis decomposition and compare parameter counts.
- Benchmark collapsed-edge GCN vs type-aware model.
- Find leakage in a metapath that includes held-out purchases.
- Add per-relation fanouts and plot sampled composition.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Heterogeneous Recommender	Uses users, movies, genres, tags	PyG HeteroData, FAISS	MovieLens	Schema + retrieval
Academic Expert Finder	Ranks authors for topics via typed paths	DGL/PyG, Elasticsearch	OGB-MAG	Large hetero benchmark
Device Fraud Graph	Links accounts, devices, IPs, payments	PyG, LightGBM, FastAPI	Synthetic/IEEE-CIS derived	Enterprise graph design

19. Quick Revision

- **Key idea:** preserve type/relation semantics during encoding and aggregation.
- **Main formula:** sum relation-specific normalized messages.
- **When to use:** entity and interaction kinds have materially different meanings.
- **Metrics:** per-type/relation task metrics and cold-start slices.
- **Traps:** ID collisions, missing reverse edges, relation imbalance/leakage.
- **Interview one-liner:** "A heterogeneous GNN translates relation-specific messages across typed feature spaces, then combines them at each target type."

20. Final Cheat Sheet

Item	Summary
Definition	Graph with multiple node/edge semantic types
Input/output	Typed features/edges -> typed embeddings/predictions
Main steps	Schema, type projection, per-relation message, combine
Hyperparameters	Type widths, relation sharing, per-relation fanout
Metrics	Per-type F1/AUC, MRR, macro score, cold start
Pros/cons	Rich semantics / schema and parameter complexity
Best uses	Recommendation, fraud, enterprise, academic networks

Knowledge Graphs

1. Overview

A knowledge graph (KG) stores facts as typed triples (**head entity**, **relation**, **tail entity**), for example (**Paris**, **capital_of**, **France**). KGs unify structured knowledge, support search and reasoning, and ground downstream AI systems. Machine learning tasks include entity/relation prediction, link prediction (knowledge-graph completion), entity alignment, question answering, and retrieval for language models.

2. Intuition

A KG is a machine-readable web of statements. If it knows "Ada wrote Paper-X" and "Paper-X concerns compilers," a system can traverse these facts to answer or retrieve evidence. Embedding models learn whether a proposed triple fits learned relational patterns, while symbolic rules preserve explicit logical constraints.

3. Prerequisites

- Directed multi-relational graphs and heterogeneous schemas
- Entity resolution, databases, triples, ontologies
- Embeddings, link prediction, ranking losses
- Basic logic, open-world assumption, and NLP entity linking

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Entity	Real/conceptual object with stable identity	Joins facts	Paris	Entity vs mention
Relation/predicate	Typed directed connection	Encodes fact semantics	capital_of	Inverse/symmetry/cardinality
Triple	Atomic fact (h, r, t)	Core storage/training unit	$(Ada, wrote, P)$	Subject-predicate-object
Ontology/schema	Classes, constraints, relation definitions	Consistency/reasoning	Person subclass Agent	KG vs property graph
Open-world assumption	Missing fact is unknown, not false	Changes negatives	unseen marriage	Negative sampling issue
Entity linking	Maps text mentions to KG IDs	Bridges unstructured text	"Apple" company/fruit	Disambiguation
Completion	Predicts missing entities/relations	Fills KG gaps	$(Paris, capital_of, ?)$	Filtered MRR
Provenance	Source/time/confidence of fact	Trust and updates	source document	Production necessity

5. Algorithm / Working Process

1. Define ontology/schema and identifiers.
2. Extract facts from databases, text, APIs, or human curation.
3. Resolve/deduplicate entities and attach provenance, qualifiers, and timestamps.
4. Validate domain/range, cardinality, and contradiction rules.
5. Store triples in a graph/RDF database or edge tables.
6. For ML, split triples carefully, generate corrupt negatives, learn entity/relation representations, and score triples.
7. At inference, retrieve/traverse facts or rank candidate entities; filter by schema and known facts.

6. Mathematical Foundation

Common knowledge-graph embedding (KGE) scores:

```

TransE:  f_r(h,t) = -||e_h + e_r - e_t||_p
DistMult: f_r(h,t) = sum_k e_hk * e_rk * e_tk
Complex: f_r(h,t) = Re(<e_h, e_r, conjugate(e_t)>)

```

TransE represents relations as translations. DistMult's diagonal bilinear score is symmetric in head/tail and cannot model antisymmetry well. ComplEx uses complex embeddings to model asymmetric relations. A margin loss is:

$$L = \sum_{(positive, negative)} \max(0, \gamma - f(positive) + f(negative))$$

or use BCE/logistic loss. For query $(h, r, ?)$, rank every candidate tail; filtered evaluation removes other known true triples. $MRR = (1/Q) \sum_q 1/rank_q$.

7. Practical Implementation

```

import torch
from torch import nn
import torch.nn.functional as F

# Triples use integer IDs: (head, relation, tail)
positive = torch.tensor([[0,0,1], [1,1,2], [2,0,3], [3,1,0]])
n_entities, n_relations, dim = 4, 2, 16

class TransE(nn.Module):
    def __init__(self):
        super().__init__()
        self.entity = nn.Embedding(n_entities, dim)
        self.relation = nn.Embedding(n_relations, dim)
        nn.init.uniform_(self.entity.weight, -.1, .1)
        nn.init.uniform_(self.relation.weight, -.1, .1)
    def score(self, triples):
        h = self.entity(triples[:, 0])
        r = self.relation(triples[:, 1])
        t = self.entity(triples[:, 2])
        return -(h + r - t).norm(p=1, dim=1) # higher is better

model = TransE(); optimizer = torch.optim.Adam(model.parameters(), lr=.02)
for _ in range(400):
    negative = positive.clone()
    negative[:, 2] = torch.randint(n_entities, (len(positive),))
    optimizer.zero_grad()
    loss = -F.logsigmoid(model.score(positive)).mean() \
           -F.logsigmoid(-model.score(negative)).mean()
    loss.backward(); optimizer.step()
    with torch.no_grad():
        model.entity.weight[:] = F.normalize(model.entity.weight, dim=1)

query = torch.tensor([[0,0,t] for t in range(n_entities)])
print("tail ranking:", model.score(query).argsort(descending=True).tolist())

```

8. Code Explanation

Entity and relation tables are trained so **head + relation** approaches **tail**. Corrupting tails creates sampled negatives, but the toy sampler may accidentally generate a known true fact; production code filters positives and often uses typed or adversarial negatives. Normalizing entity vectors prevents unbounded norms. Inference scores candidate tails and sorts descending.

9. Training / Evaluation

Choose a split consistent with the claim: random triples for transductive completion, time split for future facts, or entity-disjoint split for induction. Prevent reciprocal leakage when inverse edges are added. Generate head/tail corruptions using relation domain/range and avoid known positives. Report filtered MRR, mean rank cautiously, and Hits@1/3/10 separately for head/tail queries. Tune dimension, margin, negatives, adversarial temperature, norm, regularization, and relation model. Audit popular-entity baselines and relation cardinality groups (1-1, 1-N, N-1, N-N).

10. Complexity and Cost

KGE memory is typically $O((|E_{\text{entities}}| + |R|)d)$; large entity tables dominate. Each sampled triple score is $O(d)$. Exhaustive ranking costs $O(|\text{entities}|d)$ per query, so batching, type constraints, and ANN/retrieval stages may be required. KG construction/entity resolution can cost more than training. GNN KG models add message-passing cost over all triples.

11. Common Use Cases

- Semantic search and entity-centric retrieval
- Enterprise data integration and catalog unification

- Knowledge-graph completion and quality checks
- Question answering and explainable traversal
- RAG evidence retrieval for LLMs
- Drug discovery and biomedical relations
- Recommendations with explicit entity relations

12. Common Mistakes

- Treating missing triples as confirmed false under open world
- Random split leakage via inverse/duplicate facts
- Reporting raw rather than filtered ranking without clarity
- Using DistMult for strongly antisymmetric relations without noting limits
- Losing qualifiers, timestamps, provenance, or confidence
- Confusing entity mentions with canonical entities
- Evaluating KG extraction only by downstream anecdotes
- Letting an LLM invent facts without validation/source tracking
- Assuming embeddings perform deductive logical reasoning reliably

13. Edge Cases / Limitations

Facts may be time-dependent, contradictory, uncertain, n-ary, or context-qualified; simple triples can lose this information. Entity resolution errors cascade. Long-tail entities have few edges, embeddings memorize biases, and new entities cause cold start. KGE scores are not calibrated truth probabilities. Symbolic rules can be brittle, while neural completion may hallucinate plausible but false edges.

14. Variations

Variation	What changes	When to use	Importance
TransE/RotatE	Translation/complex rotation geometry	Relation patterns and scalable KGE	Essential
DistMult/CompLex	Real/complex bilinear score	Typed link prediction	Essential
R-GCN/CompGCN	Message passing over relations	Entity features/context	Advanced/core
Temporal KG	Facts have valid/event time	Evolving knowledge	Industry/research
RDF/property graph	Standards triples vs attributed nodes/edges	Interoperability vs app queries	System design
Neuro-symbolic KG	Combines embeddings and rules	Constraints/explainability	Research

15. Related Topics

Heterogeneous graphs are the broader typed-graph category; KGs specifically encode factual relations and often an ontology. Link prediction becomes KG completion when relation type is explicit. RAG retrieves evidence but does not itself guarantee reasoning; GraphRAG-style systems organize/retrieve connected entities and communities. Vector databases retrieve by embedding similarity, while graph databases traverse explicit relations; hybrid systems use both.

16. Interview Questions

1. **What is a knowledge graph?** A graph of canonical entities and typed factual relations, usually with schema/provenance.
2. **What is a triple?** (head/subject, relation/predicate, tail/object).
3. **Open-world assumption?** An absent fact is unknown, not necessarily false.
4. **What does TransE learn?** $e_h + e_r \approx e_t$; relations act like translations.
5. **Why does DistMult struggle with asymmetry?** Its score is unchanged when head and tail swap.
6. **What does CompLex add?** Complex-valued bilinear interactions whose conjugation permits asymmetric scores.
7. **Raw vs filtered ranking?** Filtered removes other known correct entities from corrupt candidates.
8. **Why reciprocal leakage?** A test fact can be recovered trivially if its inverse is present during training.
9. **What is entity linking?** Mapping an ambiguous mention in text to a canonical KG entity.
10. **Ontology vs KG?** Ontology defines concepts/constraints; KG contains instance facts conforming to or using them.

11. **How represent n-ary facts?** Reify an event node, use qualifiers, or a hyper-relational model.
12. **KG vs vector DB for RAG?** KG supports explicit typed traversal/provenance; vector search supports fuzzy semantic retrieval; hybrid is often strongest.
13. **Can a KGE score be read as probability?** Not without an explicit probabilistic model and calibration.
14. **How handle new entities?** Encode descriptions/attributes with an inductive model or gather relational evidence.

17. Practice Tasks

- Implement TransE and filtered MRR on a toy KG.
- Compare DistMult and ComplEx on symmetric vs antisymmetric relations.
- Build an entity-linking error analysis for ambiguous names.
- Detect inverse-edge leakage in a triple split.
- Reify a time-qualified employment fact into an event-centric schema.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Placement Skills KG	Connects roles, skills, courses, questions	Neo4j, Python, FastAPI	O*NET + curated postings	Direct domain/system value
Biomedical Completion	Predicts drug-disease/protein relations	PyKEEN/PyTorch	Hetionet/FB15k-style subset	Research + KGE evaluation
KG-grounded RAG	Retrieves paths and cited source passages	Neo4j, embeddings, LLM API	Wikidata subset + documents	Modern AI engineering

19. Quick Revision

- **Key idea:** canonical typed facts plus traversal, learning, and provenance.
- **Main formula:** TransE $\text{score} = -||h+r-t||$.
- **When to use:** explicit relationships and multi-hop/entity-centric retrieval matter.
- **Metrics:** filtered MRR, Hits@K; extraction precision/recall; QA accuracy.
- **Traps:** open-world negatives, reciprocal leakage, missing qualifiers/provenance.
- **Interview one-liner:** "A KG represents typed, sourced facts; KGE ranks plausible missing triples but does not turn plausibility into truth."

20. Final Cheat Sheet

Item	Summary
Definition	Canonical entities linked by typed factual relations
Input/output	Triples/schema/provenance -> queries, ranks, completed facts
Main steps	Extract, resolve, validate, store, embed/reason, retrieve
Hyperparameters	KGE dimension, negatives, margin, norm, relation model
Metrics	Filtered MRR/Hits@K, extraction F1, QA accuracy
Pros/cons	Explicit relations/provenance / costly curation, incompleteness
Best uses	Search, integration, QA/RAG, science, completion

Graph Transformers

1. Overview

Graph Transformers adapt Transformer attention to graph data. They can allow global all-pairs interactions or sparse/local attention while injecting graph structure through attention masks, shortest-path biases, edge encodings, Laplacian eigenvectors, random-walk features, or virtual tokens. Their main promise is stronger long-range modeling than strictly local message passing; their main challenge is $O(n^2)$ attention and the absence of a natural node order.

2. Intuition

A local GNN passes a message through every intermediary on a long path. A graph Transformer can let distant nodes communicate directly, but it must be told how they are positioned in the graph; otherwise it sees an unordered bag of node features. Structural/positional encodings provide that map.

3. Prerequisites

- Transformer query-key-value attention, residuals, LayerNorm
- GNN permutation equivariance and graph distances
- Laplacian eigenvectors and random walks
- Sparse/dense complexity and graph batching

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Global attention	Every node may attend to every node	Shortens information path	molecule-wide interaction	Quadratic cost
Structural bias	Graph relation changes attention logit	Makes connectivity visible	shortest-path bucket	Bias vs mask
Positional encoding	Node structural coordinates/features	Breaks graph symmetries	Laplacian PE	Sign/basis ambiguity
Edge encoding	Edge/path attributes affect score/value	Preserves relation semantics	bond/path types	Direct vs multi-hop edge
Graph token	Virtual node pooled by attention	Graph-level readout	[CLS] node	Bottleneck/shortcut
Sparse/hybrid attention	Restricts pairs or mixes MPNN	Scales and keeps locality	GPS layer	Long-range tradeoff
Permutation equivariance	Reordering nodes reorders outputs	Correctness requirement	$P X \rightarrow P H$	PE must transform consistently

5. Algorithm / Working Process

1. Project raw node and optional edge features into hidden states.
2. Compute graph positional/structural encodings using training-legal structure.
3. Create per-head queries, keys, and values.
4. Add graph bias/mask to scaled query-key logits.
5. Softmax over allowed source nodes and aggregate values.
6. Apply output projection, residual connection, normalization, and feed-forward network.
7. Stack layers and use node outputs, pair decoder, or graph token/pooling.
8. Optimize task loss; inference has the same attention pattern, potentially with sparse kernels.

6. Mathematical Foundation

For hidden matrix $H \in \mathbb{R}^{(n \times d)}$:

$$Q=H W_Q, K=H W_K, V=H W_V$$

$$S_{ij} = (q_i^T k_j) / \sqrt{d_k} + b_{\text{graph}}(i, j)$$

```
Attention(H) = softmax_j(S) V
```

`b_graph(i,j)` may embed shortest-path distance, edge/path features, or be `-infinity` for masked pairs. Multi-head outputs are concatenated and projected. A pre-norm block is:

```
U = H + MHA(LayerNorm(H))
H' = U + FFN(LayerNorm(U))
```

Laplacian PE uses selected nontrivial eigenvectors U_k of L ; eigenvector signs are arbitrary and repeated eigenvalues permit basis rotations. Attention is permutation equivariant when inputs/biases are permuted consistently.

7. Practical Implementation

```
import torch
from torch import nn

class GraphTransformerLayer(nn.Module):
    def __init__(self, width=16, heads=4, max_distance=5):
        super().__init__()
        self.heads, self.dk = heads, width // heads
        assert width % heads == 0
        self.qkv = nn.Linear(width, 3 * width)
        self.distance_bias = nn.Embedding(max_distance + 2, heads)
        self.out = nn.Linear(width, width)
        self.norm1, self.norm2 = nn.LayerNorm(width), nn.LayerNorm(width)
        self.ffn = nn.Sequential(nn.Linear(width, 4*width), nn.GELU(),
                                nn.Linear(4*width, width))

    def forward(self, h, distances, padding_mask=None):
        # h: [batch,n,width], distances: [batch,n,n], clipped; max+1 means unreachable
        b, n, width = h.shape
        x = self.norm1(h)
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        reshape = lambda t: t.view(b, n, self.heads, self.dk).transpose(1, 2)
        q, k, v = map(reshape, (q, k, v))
        logits = q @ k.transpose(-2, -1) / self.dk**0.5
        bias = self.distance_bias(distances).permute(0, 3, 1, 2)
        logits = logits + bias
        if padding_mask is not None:
            logits = logits.masked_fill(padding_mask[:, None, None, :], float("-inf"))
        attended = logits.softmax(-1) @ v
        attended = attended.transpose(1, 2).reshape(b, n, width)
        h = h + self.out(attended)
        return h + self.ffn(self.norm2(h))

h = torch.randn(2, 4, 16)
dist = torch.tensor([[[0,1,2,3],[1,0,1,2],[2,1,0,1],[3,2,1,0]]*2])
assert GraphTransformerLayer()(h, dist).shape == h.shape
```

8. Code Explanation

The layer performs dense multi-head scaled dot-product attention. A learned embedding converts each clipped shortest-path distance into one bias per head, so the model knows whether nodes are near, far, or unreachable. Pre-LayerNorm and residuals stabilize deeper stacks; the FFN mixes channels independently per node. A complete model adds input/edge encoders, graph padding/readout, dropout, and masked output handling.

9. Training / Evaluation

Split entire graphs or temporal structures before computing potentially leaking positional encodings. Batch graphs by similar node count to reduce padding waste. Tune layers, width, heads, FFN expansion, dropout, structural encodings, maximum distance, attention sparsity, and pooling. Compare against GNNs at matched parameter/compute budgets and report accuracy plus training memory/latency. Use scaffold splits for molecules and stress tests on larger/unseen graph sizes. Ablate positional encoding: without it, topology may be nearly invisible.

10. Complexity and Cost

Dense attention costs $O(n^2d)$ time and $O(n^2)$ attention memory per graph per head, versus message passing roughly $O(md)$ on sparse graphs. Shortest-path preprocessing can be costly ($O(n(n+m))$ via BFS for all sources in unweighted graphs). Sparse/local attention reduces cost toward $O(md)$ but weakens direct global communication. Mixed-precision, FlashAttention-style kernels, graph packing, tokens, coarsening, and linear attention can help.

11. Common Use Cases

- Molecular and materials property prediction
- Graph-level classification with long-range interactions
- Route, code, and program graphs
- Knowledge/heterogeneous graph encoders
- Combinatorial optimization
- Protein and 3-D geometric systems when combined with equivariant modules

12. Common Mistakes

- Using vanilla attention with no graph structural signal
- Treating arbitrary node ID/order as positional encoding
- Ignoring Laplacian sign and degenerate-eigenspace ambiguity
- Allocating dense attention for huge sparse graphs
- Comparing against a much smaller GNN budget
- Computing shortest-path encodings using held-out target edges
- Mixing padding nodes into softmax/readout
- Assuming global attention eliminates oversquashing or provides explanation

13. Edge Cases / Limitations

Quadratic scaling restricts graph size. Symmetric/regular graphs can remain indistinguishable without suitable encodings, yet overly strong positional encodings may memorize graph identity and hurt transfer. Disconnected graphs need an explicit unreachable encoding. Laplacian vectors can be unstable under perturbations. Global attention may focus on spurious pairs and sacrifices the strong locality bias that helps small-data regimes.

14. Variations

Variation	What changes	When to use	Importance
Graphormer	Shortest-path, centrality, edge biases	Molecular/graph benchmarks	Core interview example
GraphGPS	Local MPNN + global attention + PE	General graph learning	Modern research
SAN	Spectral attention/PE	Structural representation	Research
HGT	Type-aware attention	Heterogeneous graphs	Industry/research
Sparse/linear GT	Restricts/approximates attention	Large graphs	Systems research
Token/coarsened GT	Attends among clusters/tokens	Long graphs	Practical extension

15. Related Topics

GAT is usually edge-local additive attention; a graph Transformer often uses multi-head query-key attention with richer structural encodings and possibly global connectivity. Standard Transformers need sequential positional encodings; graph models require

permutation-consistent structural/positional encodings. Message passing supplies strong local bias; hybrid GPS-style architectures combine local GNN and global attention.

16. Interview Questions

1. **Why not use a vanilla Transformer on node features?** Without graph encodings it sees an unordered set and may not know adjacency or distance.
2. **How inject structure?** Masks, shortest-path/edge biases, Laplacian or random-walk PE, centrality, and local GNN blocks.
3. **Graph Transformer vs GAT?** GAT is generally neighbor-local additive attention; graph Transformers often use QKV attention, global/hybrid connectivity, and explicit positional structure.
4. **Dense complexity?** $O(n^2d)$ compute and $O(n^2)$ attention storage per graph/head family.
5. **Why positional encodings?** They help distinguish structural locations and graph symmetries absent from raw attributes.
6. **Problem with Laplacian PE?** Eigenvector sign ambiguity, rotations in repeated eigenspaces, instability, and preprocessing cost.
7. **How remain permutation equivariant?** Permute node states and every pairwise/positional encoding consistently; use no arbitrary ID signal.
8. **How handle disconnected nodes?** Use a dedicated unreachable-distance bucket/mask or component-aware tokens.
9. **Does global attention solve long range?** It shortens paths, but capacity, optimization, noise, and quadratic cost remain.
10. **What is GraphGPS?** A recipe combining local message passing with global attention and structural encodings.
11. **How scale?** Sparse patterns, clustering/coarsening, tokens, linear attention, efficient kernels, and size-bucketed batches.
12. **How fairly compare with GNNs?** Match parameters/compute, splits, encodings, and tuning budget; report cost as well as quality.

17. Practice Tasks

- Add adjacency-mask and distance-bias modes to the example and compare.
- Test permutation equivariance under random node reindexing.
- Train a small Graphormer-like model on molecular graphs.
- Debug NaNs caused by an all-masked attention row.
- Measure quality/memory as attention changes from local to global.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Molecular Graphormer	Predicts properties with path/edge biases	PyTorch, RDKit	ZINC/PCQM subset	Modern architecture + chemistry
GNN-vs-GT Benchmark	Controls parameter and compute budgets	PyG, MLflow	Long Range Graph Benchmark	Research rigor
Sparse Code Graph Transformer	Detects vulnerable functions in AST/dataflow graph	PyTorch, tree-sitter	Devign subset	AI engineering + program analysis

19. Quick Revision

- **Key idea:** attention plus permutation-consistent graph structure/position.
- **Main formula:** $\text{softmax}(QK^T/\sqrt{d}) + B_{\text{graph}}V$.
- **When to use:** long-range graph interactions justify extra cost.
- **Metrics:** task quality, memory, latency, scaling/generalization.
- **Traps:** no structural encoding, quadratic memory, PE leakage/ambiguity.
- **Interview one-liner:** "A graph Transformer is not merely attention on nodes; its structural bias and positional encoding define what graph it sees."

20. Final Cheat Sheet

Item	Summary
Definition	Transformer attention adapted to unordered graph structure
Input/output	Node/edge/structural encodings -> node/graph predictions
Main steps	Encode, QKV, graph bias/mask, attention, FFN, readout
Hyperparameters	Layers, heads, width, PE, distance buckets, sparsity
Metrics	Task metric + memory/latency/size transfer
Pros/cons	Direct long-range interaction / quadratic and PE complexity
Best uses	Medium graphs with important global dependencies

Geometric Deep Learning

1. Overview

Geometric deep learning (GDL) is the symmetry- and structure-aware framework that generalizes deep learning from Euclidean grids and sequences to graphs, manifolds, meshes, point clouds, molecules, and physical systems. It asks: what transformations of the input should leave an output unchanged (invariance) or transform it predictably (equivariance), and how can architectures encode those priors? CNNs, GNNs, spherical CNNs, and E(3)-equivariant networks fit under this umbrella.

2. Intuition

A rotated molecule is the same molecule and should have the same predicted energy, but its force vectors must rotate with it. Rather than forcing a generic network to relearn this from every rotated example, an invariant/equivariant architecture builds the rule into the model. This improves data efficiency, correctness, and generalization.

3. Prerequisites

- Linear algebra, vectors/matrices, norms, eigenvalues
- Basic group actions: permutations, translations, rotations, reflections
- Invariance vs equivariance and neural network fundamentals
- Graphs, manifolds/meshes or point clouds at an intuitive level
- Calculus/gradients for physical tasks; representation theory for research depth

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Domain	Structured space supporting data	Determines locality/symmetry	grid, graph, sphere	Why non-Euclidean?
Group action	Transformation applied to data	Formalizes symmetry	rotation/permutation	Group G vs graph G context
Invariance	Output unchanged	Scalar/global properties	molecular energy	$f(Tx)=f(x)$
Equivariance	Output transforms predictably	Structured/vector outputs	force rotates	$f(Tx)=T'f(x)$
Locality	Nearby elements interact	Efficiency/generalization	mesh neighbors	Geodesic vs Euclidean
Weight sharing	Same operator across symmetric locations	Reduces parameters	convolution	Symmetry creates sharing

Subtopic	Meaning	Why it matters	Example	Interview angle
Coordinates/features	Geometry plus attributes	Determines valid operations	atom position/type	Avoid coordinate-frame leakage
Gauge/local frame	Choice of local coordinates	Required on curved domains	tangent frames	Research-level concept

5. Algorithm / Working Process

1. Identify the domain: graph, set, point cloud, mesh, manifold, or group.
2. Identify transformations under which the problem is invariant/equivariant: permutations, translations, rotations, reflections, or local-frame changes.
3. Choose representations for scalar, vector, or higher-order features.
4. Construct local/shared operators using invariant quantities (distances, angles) and equivariant updates where outputs transform.
5. Compose layers; equivariance is preserved under compatible compositions and nonlinearities.
6. Apply invariant pooling for global scalar targets or retain equivariant outputs for vectors/fields.
7. Train with task loss, validate transformed-input consistency, and evaluate physical/geometric generalization.

6. Mathematical Foundation

Let group element g act on input through representation $\rho_{\text{in}}(g)$ and output through $\rho_{\text{out}}(g)$:

Invariant: $f(\rho_{\text{in}}(g)x) = f(x)$
 Equivariant: $f(\rho_{\text{in}}(g)x) = \rho_{\text{out}}(g) f(x)$

Permutation-equivariant graph layers satisfy $f(PX, PAP^T) = P f(X, A)$. A graph readout is invariant: $R(PH) = R(H)$. For 3-D coordinates r_i , $E(3)$ transformations are $r_i' = Q r_i + t$, with orthogonal Q (rotation/reflection). Pair distance is invariant:

$$||r_i' - r_j'|| = ||Q(r_i - r_j)|| = ||r_i - r_j||$$

An equivariant coordinate/message update can use:

```
m_ij = phi_e(h_i, h_j, ||r_i - r_j||^2, e_ij)
h_i' = phi_h(h_i, sum_j m_ij)
r_i' = r_i + sum_j (r_i - r_j) phi_x(m_ij)
```

The scalar coefficient is invariant, while relative vectors rotate/reflection-transform, making the coordinate update $E(n)$ -equivariant. For conservative physical systems, forces often follow $F_i = -\partial E / \partial r_i$.

7. Practical Implementation

```
import torch
from torch import nn

class InvariantPointEnergy(nn.Module):
    """Translation/rotation/reflection-invariant scalar for a point cloud."""
    def __init__(self, hidden=32):
        super().__init__()
        self.pair = nn.Sequential(nn.Linear(1, hidden), nn.SiLU(),
                                   nn.Linear(hidden, hidden), nn.SiLU())
        self.readout = nn.Linear(hidden, 1)
```

```
def forward(self, positions, batch):
    # Small-graph reference: all within-graph unordered pairs.
    graph_features = []
    for graph_id in range(int(batch.max()) + 1):
        r = positions[batch == graph_id]
        i, j = torch.triu_indices(len(r), len(r), offset=1)
        squared_distance = ((r[i] - r[j]) ** 2).sum(1, keepdim=True)
        graph_features.append(self.pair(squared_distance).sum(0))
    return self.readout(torch.stack(graph_features)).squeeze(1)

torch.manual_seed(0)
r = torch.randn(5, 3); batch = torch.tensor([0,0,0,1,1])
model = InvariantPointEnergy()
angle = torch.tensor(.7)
Q = torch.tensor([[torch.cos(angle), -torch.sin(angle), 0],
                  [torch.sin(angle), torch.cos(angle), 0], [0,0,1.]])
shift = torch.tensor([2., -1., .5])
e1 = model(r, batch)
e2 = model(r @ Q.T + shift, batch)
assert torch.allclose(e1, e2, atol=1e-5)
```

8. Code Explanation

The model uses only squared pairwise distances, which remain unchanged under translation, rotation, and reflection. A shared MLP embeds each pair, sum pooling is invariant to point order and pair order, and a readout predicts one scalar per point cloud. The assertion is a property test of E(3) invariance. The model is intentionally small-graph and loses orientation/chirality; practical molecular networks use neighbor cutoffs, atom features, and equivariant/angular information.

9. Training / Evaluation

Split by structures, trajectories, time, or molecular scaffolds—not individual highly correlated conformations. Standardize scalar targets using training statistics and conserve units. Use MAE/RMSE for energies, force MAE for vectors, trajectory stability for simulation, and task metrics for classification. In addition, test symmetry error: transform inputs and compare outputs against the required transformed result. Tune cutoff radius, radial basis size, layers, hidden irreducible representations/orders, neighbors, and loss balance between energy and forces. Evaluate out-of-distribution sizes, compositions, rotations, and geometries.

10. Complexity and Cost

All-pairs point interactions are $O(n^2)$; radius/k-nearest-neighbor graphs reduce this toward $O(m)$, plus neighbor-search cost. Higher-order tensor/spherical harmonic features and Clebsch-Gordan products can be substantially more expensive than scalar GNNs. Memory grows with edges, channels, representation orders, and layers. GPUs are usually needed for large molecular/physical datasets; CPU neighbor construction can bottleneck. Symmetry often pays back through data efficiency.

11. Common Use Cases

- Molecular energy, forces, dynamics, and property prediction
- Protein structure and interaction modeling
- Point-cloud classification, segmentation, and registration
- Mesh and fluid/solid simulation
- Weather/climate modeling on spheres
- Robotics, poses, and 3-D perception
- Graph learning as permutation geometric deep learning

12. Common Mistakes

- Confusing invariance with equivariance
- Predicting a vector with an invariant-only architecture
- Feeding absolute coordinates when translation should not matter
- Using distance-only features when chirality/orientation matters

- Applying arbitrary nonlinearities to vector/tensor components and breaking equivariance
- Claiming symmetry from data augmentation without testing it
- Randomly splitting correlated trajectory frames/conformers
- Ignoring units, boundary conditions, and conservation laws
- Building a neighbor graph after transformation inconsistently

13. Edge Cases / Limitations

The required symmetry may be approximate or broken by external fields, sensors, coordinate frames, boundaries, or gravity; enforcing too much symmetry discards real signal. Distance-only models cannot distinguish mirror images. Discrete neighbor cutoffs can cause discontinuities unless smoothed. Equivariant models are mathematically and computationally complex, and perfect one-step errors do not ensure stable long rollouts.

14. Variations

Variation	What changes	When to use	Importance
Deep Sets	Permutation-invariant set processing	Point sets without relations	Foundational
PointNet/PointNet++	Shared point MLP + pooling/local hierarchy	Point clouds	CV placements
EGNN	E(n)-equivariant scalar messages/coordinate updates	Efficient 3-D graphs	Core modern
SE(3)-Transformer	Rotation-equivariant attention/tensor features	Rich 3-D interactions	Research
NequIP/MACE	E(3)-equivariant interatomic potentials	Accurate molecular dynamics	Research/industry
Spherical CNN	Convolution respecting spherical rotations	Climate/3-D shapes	Specialized
Mesh/Manifold CNN	Local operators on curved surfaces	Geometry/physics	Research

15. Related Topics

GNNs are GDL models equivariant to node permutations. CNNs are translation-equivariant on regular grids; data augmentation approximates symmetry statistically, whereas equivariant networks enforce it architecturally. Invariant models suit scalar properties; equivariant models suit coordinates, vectors, and fields. Graph Transformers become geometric when their encodings/attention respect the domain's transformations.

16. Interview Questions

1. **What is geometric deep learning?** Learning on structured/non-Euclidean domains using their symmetries and geometry as inductive biases.
2. **Invariance vs equivariance?** Invariant output stays fixed; equivariant output transforms predictably with input.
3. **Give a molecular example.** Energy is rotation/translation invariant; force vectors are rotation equivariant and translation invariant.
4. **How is a GNN geometric?** It respects node-permutation equivariance through shared messages and invariant aggregation.
5. **Why are distances invariant?** Orthogonal rotations/reflections preserve Euclidean norm, and differences cancel translation.
6. **What is E(3) vs SE(3)?** E(3) includes translations, rotations, and reflections; SE(3) includes translations and proper rotations, not reflections.
7. **Why does chirality matter?** Mirror images can have identical distances yet different chemistry; E(3)-invariant distance-only models cannot distinguish them.
8. **Architecture symmetry vs augmentation?** Architecture guarantees the property for all represented transformations; augmentation only encourages it on sampled examples.
9. **Why can componentwise ReLU break vector equivariance?** Rotating a vector and applying coordinatewise nonlinearity need not equal applying it then rotating.
10. **What is a group action?** A consistent mapping that applies each symmetry group element to data/features.
11. **How test equivariance?** Transform input, run both paths, transform the original output, and measure their discrepancy.

12. **What are irreducible representations?** Basic transformation types into which group representations decompose; they organize scalar/vector/higher-order channels.
13. **When should symmetry not be imposed?** When external frames/fields or task semantics genuinely break it.
14. **Why use cutoff neighborhoods?** Physical interactions are often local and all-pairs scaling is expensive; long-range forces may need separate treatment.

17. Practice Tasks

- Write property tests for permutation, translation, and rotation behavior.
- Predict point-cloud scalar properties using invariant distances.
- Train energy-only vs joint energy-force models on a molecular subset.
- Debug a vector network whose componentwise activation breaks rotation equivariance.
- Extend the example with atom types and a smooth radius cutoff.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Equivariant Molecule Predictor	Predicts energies/forces and verifies symmetry	PyTorch, e3nn/PyG, ASE	QM9/MD17 subset	Strong research signal
Point Cloud Shape Classifier	Compares PointNet and graph/equivariant encoders	PyTorch, Open3D	ModelNet40	CV + geometry breadth
Mesh Dynamics Simulator	Learns deformation/flow rollouts on meshes	PyG, PyTorch, Plotly	Synthetic cloth/mesh	Physics + rollout evaluation

19. Quick Revision

- **Key idea:** encode domain symmetries so predictions transform correctly.
- **Main formula:** $f(\text{rho_in}(g)x) = \text{rho_out}(g)f(x)$.
- **When to use:** graphs, sets, 3-D, manifolds, meshes, and physics.
- **Metrics:** task error plus invariance/equivariance error and OOD stability.
- **Traps:** wrong symmetry, invariant/equivariant confusion, correlated splits.
- **Interview one-liner:** "Geometric deep learning replaces arbitrary coordinate dependence with symmetry-aware shared operators on structured domains."

20. Final Cheat Sheet

Item	Summary
Definition	Deep learning respecting geometry and transformation groups
Input/output	Structured domain/features -> invariant or equivariant prediction
Main steps	Identify symmetry, choose representations, local operators, readout
Hyperparameters	Cutoff, neighbors, layers, channels, tensor order, loss weights
Metrics	MAE/RMSE/task score + symmetry error/OOD rollout
Pros/cons	Data-efficient/physically consistent / complex and costly
Best uses	Graphs, molecules, point clouds, meshes, physical systems

Cross-Topic Comparison for Interviews

Method/topic	Learns from	Inductive by default?	Main strength	Main limitation
DeepWalk	Uniform walk co-occurrence	No	Simple scalable structural embedding	No features/cold start
Node2Vec	Biased walk co-occurrence	No	Tunable local/outward exploration	p, q tuning; transductive
GCN	Features + normalized adjacency	Model can transfer, standard setup often transductive	Simple strong homophily baseline	Oversmoothing/full-batch scaling
GraphSAGE	Features + sampled neighbors	Yes	Large-graph and unseen-node inference	Sampling variance/fanout
GAT	Features + learned edge weights	Yes if feature-based	Adaptive neighbor importance	Extra memory; attention is not explanation
Heterogeneous GNN	Typed features and relations	Architecture-dependent	Preserves schema semantics	Parameter/sampling complexity
KGE	Entity/relation lookup vectors	Usually no	Efficient typed triple ranking	Open-world negatives/cold start
Graph Transformer	Features + global/sparse attention + graph PE	Architecture-dependent	Long-range interaction	Quadratic dense attention
Geometric/equivariant model	Coordinates/structure + symmetry	Often yes	Correct physical transformation behavior	Mathematical/compute complexity

Suggested Placement Study Order

1. Master graph representation, sparse adjacency, degree, Laplacian, and split leakage.
2. Implement node classification and link prediction baselines.
3. Understand DeepWalk and Node2Vec as Skip-gram over random walks.
4. Derive message passing, then GCN, GraphSAGE, and GAT differences.
5. Learn heterogeneous schemas and KG completion/evaluation.
6. Study graph positional encodings, graph Transformers, and geometric symmetry.
7. Build one project with a correct deployment-style split and a non-graph baseline.

The most valuable interview habit is to start from the task and data contract: identify the prediction unit, what graph information exists at inference, how the split prevents leakage, and which baseline proves that graph structure adds signal. Architecture choice comes after those answers.